Smart Hotel MAS

4-Memory Architecture Workshop

2-Hour Hands-On: Multi-Agent System with Working + Episodic + Semantic + Knowledge Graph Memory

CrewAI | Neo4j | ChromaDB | SQLite | PuLP | Stable-Baselines3

Agentic AI Coding Fitness • Week 9 | AltoTech Global

agentic-coding-fitness.altotech.ai • AltoTech Global

Memory Layers: L1 Working (dict) → L2 Episodic (SQLite) → L3 Semantic (ChromaDB) → L4 KG (Neo4j)

Prerequisites: mas_environment_setup.pdf • mas_kg_coding.pdf • Docker + Neo4j + ChromaDB

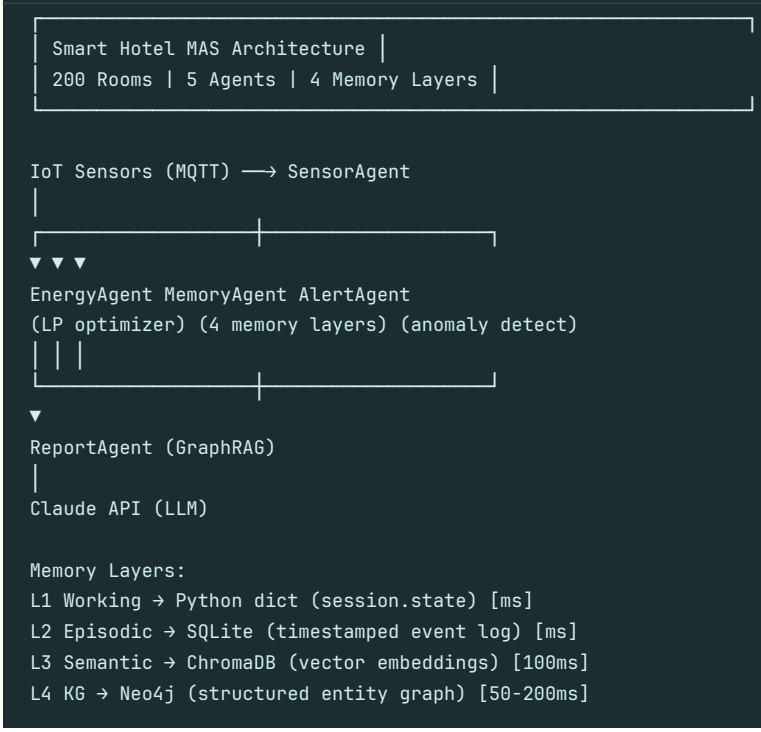
Pre-Workshop Setup

Get Docker + dependencies running before the session starts

System Architecture

The Smart Hotel MAS uses 5 specialized agents coordinated through a 4-layer memory architecture. Each layer serves a different access pattern and retention horizon.

Smart Hotel MAS — System Architecture



Docker Compose Setup

docker-compose.yml

```
# docker-compose.yml — Smart Hotel MAS Stack
version: "3.8"
services:
neo4j:
image: neo4j:5.18-community
ports:
- "7474:7474" # Browser UI
- "7687:7687" # Bolt
environment:
NEO4J_AUTH: neo4j/hotel_mas_2024
NEO4J_PLUGINS: '["apoc","graph-data-science"]'
NEO4J_dbms_memory_heap_max__size: 2G
volumes:
- neo4j_data:/data
healthcheck:
test: ["CMD", "neo4j", "status"]
interval: 10s

chromadb:
image: chromadb/chroma:latest
ports: ["8001:8000"]
volumes: [chroma_data:/chroma/chroma]
```

```
redis:
image: redis:7-alpine
ports: ["6379:6379"]

volumes:
neo4j_data:
chroma_data:
```

pip install

```
# Python dependencies
pip install neo4j chromadb crewai crewai-tools \
langchain langchain-community langchain-anthropic \
langchain-experimental \
pulp stable-baselines3 gymnasium \
scikit-learn pandas numpy \
prophet redis sqlite3
```

Hotel Domain Model

Entity	Description	Key Properties	Neo4j Label
Room	Hotel room	id, floor, type, capacity, rate_thb, status	:Room
Device	IoT sensor/actuator	id, type, room_id, model, calibration_date	:Device
SensorReading	IoT measurement	temp_c, humidity, occupancy, kwh, ts	:SensorReading
Alert	Triggered threshold	type, severity, threshold, value, ts	:Alert
MaintenanceJob	Repair task	type, priority, status, assigned_to, ts	:MaintenanceJob
Staff	Hotel employee	id, name, role, shift, skills	:Staff
Guest	Room guest	id, checkin, checkout, preferences	:Guest
EnergyEvent	Optimization action	action, setpoint, saving_kwh, ts	:EnergyEvent
Agent	MAS agent	id, role, last_active	:Agent
Event	Agent action log	type, details, ts	:Event

Seed Data Generator

Run this script once to populate Neo4j with 200 rooms, 400 devices, and 30 days of sensor history before the workshop.

seed_hotel.py

```
# seed_hotel.py — Run once to populate Neo4j with sample data
import random
from datetime import datetime, timedelta
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "hotel_mas_2024")
)

def seed_database():
```

```
with driver.session() as s:
# Schema
s.run("CREATE CONSTRAINT IF NOT EXISTS "
"FOR (r:Room) REQUIRE r.id IS UNIQUE")
s.run("CREATE CONSTRAINT IF NOT EXISTS "
"FOR (d:Device) REQUIRE d.id IS UNIQUE")
s.run("CREATE INDEX IF NOT EXISTS "
"FOR (sr:SensorReading) ON (sr.ts)")

# Create 200 rooms across 5 floors
for floor in range(1, 6):
for room_num in range(1, 41):
room_id = f"R{floor}{room_num:02d}"
room_type = random.choice(
["standard","deluxe","suite"]
)
s.run("""
MERGE (r:Room {id: $rid})
SET r.floor = $floor,
r.type = $rtype,
r.capacity = $cap,
r.rate_thb = $rate,
r.status = 'active'
""", rid=room_id, floor=floor,
rtype=room_type,
cap=2 if room_type=='suite' else 1,
rate=random.randint(2000,8000))

# Each room gets HVAC + sensor
for dtype in ["HVAC", "SENSOR"]:
dev_id = f"{dtype}_{room_id}"
s.run("""
MERGE (d:Device {id: $did})
SET d.type = $dtype,
d.room_id = $rid,
d.status = 'active'
WITH d
MATCH (r:Room {id: $rid})
MERGE (r)-[:HAS_DEVICE]->(d)
""", did=dev_id, dtype=dtype, rid=room_id)

# Seed 30 days of readings (10 per room)
base_date = datetime.now() - timedelta(days=30)
for floor in range(1, 6):
for room_num in range(1, 41):
room_id = f"R{floor}{room_num:02d}"
for i in range(10):
ts = base_date + timedelta(
days=random.randint(0,29),
hours=random.randint(0,23)
)
temp = round(20 + random.gauss(4, 2), 1)
s.run("""
MATCH (d:Device {
id: $did, type: 'SENSOR'})
CREATE (sr:SensorReading {
ts: datetime($ts),
temp_c: $temp,
humidity: $hum,
occupancy: $occ,
kwh: $kwh
})
CREATE (d)-[:RECORDED]->(sr)
```

```
""", did=f"SENSOR_{room_id}",
ts=ts.isoformat(),
temp=temp,
hum=round(40+random.random()*30,1),
occ=random.choice([True,False]),
kwh=round(1+random.random()*3,2))

print("Seeded: 200 rooms, 400 devices, 2000 readings")

if __name__ == "__main__":
seed_database()
driver.close()
```

Theory Block (30 Minutes)

Why 4 memories? Hotel KG schema. Mathematical models. Agent interactions.

■ 0:00–0:30 — Theory

Why 4 Memory Types?

Cognitive Science Analogy

Human memory has four levels — sensory (working), short-term episodic, long-term semantic, and procedural (structured knowledge). Agent memory mirrors this: working memory holds current task state, episodic stores what happened, semantic stores what things mean, and knowledge graph stores how things relate.

Memory Type	Human Analogy	Agent Implementation	Hotel Use Case	Access Time
L1: Working	RAM / attention window	Python dict, ADK session.state	Current room readings in-context	<1ms
L2: Episodic	Short-term memory / diary	SQLite timestamped event log	History of HVAC actions taken	<10ms
L3: Semantic	Long-term knowledge / expertise	ChromaDB vector embeddings	Find similar past incidents	50-200ms
L4: Knowledge Graph	Procedural / relational knowledge	Neo4j Cypher traversal	Which HVAC units caused >3 alerts?	50-200ms

Hotel Knowledge Graph Schema Design

The hotel KG uses 10 node labels and 15 relationship types. Constraints ensure data integrity; indexes accelerate time-range queries.

Hotel KG Schema Setup

```
// Hotel KG Schema — 10 node labels, 15 relationship types
// Run this setup script once (seeds schema + constraints)

// === CONSTRAINTS (uniqueness) ===
CREATE CONSTRAINT IF NOT EXISTS FOR (r:Room)
REQUIRE r.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (d:Device)
REQUIRE d.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (s:Staff)
REQUIRE s.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (g:Guest)
REQUIRE g.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (a:Agent)
REQUIRE a.id IS UNIQUE;

// === INDEXES ===
CREATE INDEX IF NOT EXISTS FOR (sr:SensorReading)
ON (sr.ts);
CREATE INDEX IF NOT EXISTS FOR (al:Alert)
ON (al.ts, al.severity);
CREATE INDEX IF NOT EXISTS FOR (e:Event)
ON (e.type, e.ts);

// === RELATIONSHIP TYPES ===
// (:Room)-[:HAS_DEVICE]->(:Device)
```

```
// (:Device)-[:RECORDED]->(:SensorReading)
// (:SensorReading)-[:TRIGGERED]->(:Alert)
// (:Alert)-[:RESOLVED_BY]->(:MaintenanceJob)
// (:MaintenanceJob)-[:ASSIGNED_TO]->(:Staff)
// (:Room)-[:OCCUPIED_BY]->(:Guest)
// (:Agent)-[:PERFORMED]->(:Event)
// (:Event)-[:INVOLVES]->(:Room|Device|Alert|Guest)
// (:EnergyEvent)-[:APPLIED_TO]->(:Room)
// (:Room)-[:ON_FLOOR]->(:Floor)
// (:Room)-[:ADJACENT_TO]->(:Room)
// (:Device)-[:MAINTAINED_BY]->(:Staff)
// (:Alert)-[:CAUSED_BY]->(:Device)
// (:Guest)-[:CHECKED_INTO]->(:Room)
// (:Staff)-[:SKILLED_IN]->(:Skill)
```

Mathematical Models Overview

Model	Problem	Key Variables	Library	When to Call
Linear Programming (PuLP)	Minimize energy cost while satisfying comfort constraints	Decision: HVAC setpoints; Constraints: temp bounds, budget; Objective: sum(kWh x tariff)	pulp	Every 15 minutes based on occupancy forecast
ARIMA Forecasting (Prophet)	Predict room occupancy for next 4 hours	Input: 30-day occupancy history; Output: occupancy_probability[t+1..t+4]	prophet	Hourly, feeds LP constraints
Q-Learning RL (Stable-Baselines3)	Learn optimal HVAC policy from experience	State: (temp, occupancy, time_of_day); Action: setpoint delta; Reward: -energy + comfort	stable-baselines3	Continuous background learning
Isolation Forest (scikit-learn)	Detect anomalous sensor readings in real-time	Features: temp_c, humidity, kwh, occupancy; Contamination: 0.05	scikit-learn	Per reading (online inference)

Agent Interaction Protocol

The agents communicate via shared memory layers. Each agent writes to appropriate memory layers and reads from layers it needs.

Agent Interaction Timeline

```
Time 0:00 SensorAgent reads IoT data
↓ stores to L1 (working memory dict)
↓ stores to L2 (SQLite episodic log)

Time 0:01 MemoryAgent receives sensor batch
↓ stores events to L4 (Neo4j KG)
↓ embeds reading text to L3 (ChromaDB)

Time 0:02 EnergyAgent queries KG for occupancy pattern
↓ runs ARIMA forecast (next 4h occupancy)
↓ runs LP optimizer (optimal HVAC setpoints)
↓ sends actions to SensorAgent (HVAC control)

Time 0:03 AlertAgent queries L3 (semantic: similar anomalies?)
↓ runs Isolation Forest on current readings
↓ if anomaly: stores Alert node in L4 (Neo4j)
↓ queries L4: which maintenance staff available?
↓ creates MaintenanceJob node + ASSIGNED_TO edge

Time 0:05 ReportAgent uses GraphRAG (Claude + Neo4j)
```

```
↓ traverses KG for shift summary
↓ generates natural language hotel status report
```

Hands-On Block (90 Minutes)

6 Checkpoints — each 15 minutes

■ 0:30–0:45 CHECKPOINT 1: Neo4j Schema + Seed Data

Goal: Have a running Neo4j instance with 200 rooms, 400 devices, and 30 days of sensor history. Verify with 3 check queries.

checkpoint1_seed.py

```
# checkpoint1_seed.py — Run this to set up Neo4j schema and seed data
import random
from datetime import datetime, timedelta
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "hotel_mas_2024")
)

def setup_schema(s):
s.run("CREATE CONSTRAINT IF NOT EXISTS "
"FOR (r:Room) REQUIRE r.id IS UNIQUE")
s.run("CREATE CONSTRAINT IF NOT EXISTS "
"FOR (d:Device) REQUIRE d.id IS UNIQUE")
s.run("CREATE CONSTRAINT IF NOT EXISTS "
"FOR (st:Staff) REQUIRE st.id IS UNIQUE")
s.run("CREATE CONSTRAINT IF NOT EXISTS "
"FOR (g:Guest) REQUIRE g.id IS UNIQUE")
s.run("CREATE INDEX IF NOT EXISTS "
"FOR (sr:SensorReading) ON (sr.ts)")
s.run("CREATE INDEX IF NOT EXISTS "
"FOR (al:Alert) ON (al.ts, al.severity)")
print("Schema constraints and indexes created.")

def seed_rooms(s):
for floor in range(1, 6):
for room_num in range(1, 41):
room_id = f"R{floor}{room_num:02d}"
room_type = random.choice(
["standard", "deluxe", "suite"]
)
s.run("""
MERGE (r:Room {id: $rid})
SET r.floor = $floor,
r.type = $rtype,
r.capacity = $cap,
r.rate_thb = $rate,
r.status = 'active'
""", rid=room_id, floor=floor,
rtype=room_type,
cap=2 if room_type == 'suite' else 1,
rate=random.randint(2000, 8000))

for dtype in ["HVAC", "SENSOR"]:
dev_id = f"{dtype}_{room_id}"
s.run("""
MERGE (d:Device {id: $did})
SET d.type = $dtype,
d.room_id = $rid,
```

```
d.status = 'active'
WITH d
MATCH (r:Room {id: $rid})
MERGE (r)-[:HAS_DEVICE]->(d)
""", did=dev_id, dtype=dtype, rid=room_id)

def seed_readings(s):
base_date = datetime.now() - timedelta(days=30)
for floor in range(1, 6):
for room_num in range(1, 41):
room_id = f"R{floor}{room_num:02d}"
for i in range(10):
ts = base_date + timedelta(
days=random.randint(0, 29),
hours=random.randint(0, 23)
)
temp = round(20 + random.gauss(4, 2), 1)
s.run("""
MATCH (d:Device {
id: $did, type: 'SENSOR'})
CREATE (sr:SensorReading {
ts: datetime($ts),
temp_c: $temp,
humidity: $hum,
occupancy: $occ,
kwh: $kwh
})
CREATE (d)-[:RECORDED]->(sr)
""", did=f"SENSOR_{room_id}",
ts=ts.isoformat(), temp=temp,
hum=round(40 + random.random() * 30, 1),
occ=random.choice([True, False]),
kwh=round(1 + random.random() * 3, 2))

def seed_database():
with driver.session() as s:
setup_schema(s)
seed_rooms(s)
seed_readings(s)
print("Seeded: 200 rooms, 400 devices, 2000 readings")

if __name__ == "__main__":
seed_database()
driver.close()
```

Verification Queries — run these in Neo4j Browser to confirm the seed data:

Verification Queries

```
-- Verify: Count entities
MATCH (r:Room) RETURN count(r) AS rooms;
-- Expected: 200

MATCH (d:Device) RETURN count(d) AS devices;
-- Expected: 400

MATCH (sr:SensorReading) RETURN count(sr) AS readings;
-- Expected: 2000
```

Checkpoint 1 Complete

Neo4j Browser at http://localhost:7474 should show the graph. Run CALL db.schema.visualization() to see the full schema diagram. All 3 count queries should return the expected values before proceeding.

■ 0:45–1:00 CHECKPOINT 2: Working + Episodic Memory

Goal: Implement L1 (WorkingMemory) and L2 (EpisodicMemory). Understand the speed/durability trade-off between in-process dict and SQLite.

L1: Working Memory (in-context dict)

checkpoint2_memory.py

```
# checkpoint2_memory.py — Working + Episodic Memory
import sqlite3
import json
from datetime import datetime
from typing import Any

# ── L1: Working Memory (in-context dict) ──────────────────
class WorkingMemory:
"""
Ephemeral memory for the current agent task.
Lives in Python process — lost at end of run.
"""
def __init__(self):
self._state: dict = {}

def set(self, key: str, value: Any):
self._state[key] = value

def get(self, key: str, default=None) -> Any:
return self._state.get(key, default)

def get_all(self) -> dict:
return dict(self._state)

def clear(self):
self._state.clear()

def summary(self) -> str:
"""Compact string for LLM context injection."""
items = [f"{k}={v}" for k, v in self._state.items()]
return "Working Memory: {" + ", ".join(items[:10]) + "}"


# ── L2: Episodic Memory (SQLite log) ──────────────────────
class EpisodicMemory:
"""
Timestamped event log stored in SQLite.
Persists across runs. Good for 'what happened today?'
"""
def __init__(self, db_path: str = "hotel_episodes.db"):
self.conn = sqlite3.connect(db_path)
self._create_table()

def _create_table(self):
self.conn.execute("""
CREATE TABLE IF NOT EXISTS episodes (
id INTEGER PRIMARY KEY AUTOINCREMENT,
ts TEXT NOT NULL,
agent_id TEXT NOT NULL,
event_type TEXT NOT NULL,
data TEXT NOT NULL,
tags TEXT DEFAULT ''
)
""")
```

```
self.conn.execute(
"CREATE INDEX IF NOT EXISTS ep_ts_idx "
"ON episodes (ts)"
)
self.conn.commit()

def store(self, agent_id: str,
event_type: str,
data: dict,
tags: list = None):
"""Log an agent event to SQLite."""
self.conn.execute("""
INSERT INTO episodes
(ts, agent_id, event_type, data, tags)
VALUES (?, ?, ?, ?, ?)
""", (
datetime.now().isoformat(),
agent_id,
event_type,
json.dumps(data),
','.join(tags or [])
))
self.conn.commit()

def recall_recent(self, hours: int = 24,
event_type: str = None,
limit: int = 20) -> list:
"""Return recent episodes, optionally filtered."""
from datetime import timedelta
cutoff = (datetime.now()
- timedelta(hours=hours)).isoformat()
if event_type:
cursor = self.conn.execute("""
SELECT ts, agent_id, event_type, data
FROM episodes
WHERE ts > ? AND event_type = ?
ORDER BY ts DESC LIMIT ?
""", (cutoff, event_type, limit))
else:
cursor = self.conn.execute("""
SELECT ts, agent_id, event_type, data
FROM episodes
WHERE ts > ?
ORDER BY ts DESC LIMIT ?
""", (cutoff, limit))
return [
{"ts": r[0], "agent": r[1],
"type": r[2], "data": json.loads(r[3])}
for r in cursor.fetchall()
]

def recall_by_room(self, room_id: str,
limit: int = 10) -> list:
cursor = self.conn.execute("""
SELECT ts, event_type, data FROM episodes
WHERE data LIKE ?
ORDER BY ts DESC LIMIT ?
""", (f'%{room_id}%', limit))
return [
{"ts": r[0], "type": r[1],
"data": json.loads(r[2])}
for r in cursor.fetchall()
]
```

```
def close(self):
self.conn.close()


# ── Test ──────────────────────────────────────────────
if __name__ == "__main__":
wm = WorkingMemory()
wm.set("current_batch", ["R101","R102","R103"])
wm.set("batch_start_ts", datetime.now().isoformat())
print(wm.summary())

em = EpisodicMemory()
em.store("SensorAgent", "sensor_batch_read", {
"rooms": ["R101","R102"],
"avg_temp": 24.5,
"anomalies": 0
})
em.store("AlertAgent", "alert_triggered", {
"room": "R101",
"type": "HIGH_TEMP",
"value": 28.3
})

recent = em.recall_recent(hours=1)
print(f"Recent episodes: {len(recent)}")
for ep in recent:
print(f" [{ep['ts'][:19]}] {ep['agent']}: "
f"{ep['type']}")
em.close()
```

Key Insight: L1 (WorkingMemory) is lost when the Python process ends — perfect for task-level state. L2 (EpisodicMemory) persists to SQLite — perfect for "what did we do today?" recall. Use both together: L1 for speed, L2 for durability.

■ 1:00–1:15 CHECKPOINT 3: Semantic Memory (ChromaDB)

Goal: Implement L3 SemanticMemory using ChromaDB. Store sensor readings as text embeddings and retrieve similar past incidents.

checkpoint3_semantic.py

```
# checkpoint3_semantic.py — Semantic Memory with ChromaDB
import chromadb
from chromadb.utils import embedding_functions
import json
from datetime import datetime

class SemanticMemory:
"""
L3: Vector-based semantic memory.
Stores: sensor readings, incidents, decisions as text.
Retrieves: 'find similar past incidents to current reading'
"""
def __init__(self,
host: str = "localhost",
port: int = 8001,
collection: str = "hotel_memory"):
self.client = chromadb.HttpClient(
host=host, port=port
)
# Use sentence-transformers for embeddings
# (free, runs locally — no API key needed)
self.embedder = (
embedding_functions
.SentenceTransformerEmbeddingFunction(
model_name="all-MiniLM-L6-v2"
)
)
self.collection = self.client.get_or_create_collection(
name=collection,
embedding_function=self.embedder
)

def store_reading(self, room_id: str,
reading: dict,
event_type: str = "sensor_reading"):
"""Convert sensor reading to text + embed + store."""
doc_text = (
f"Room {room_id}: temp={reading.get('temp_c')}°C, "
f"humidity={reading.get('humidity')}%, "
f"occupancy={reading.get('occupancy')}, "
f"energy={reading.get('kwh')} kWh, "
f"event={event_type}, "
f"time={datetime.now().strftime('%H:%M')}"
)
self.collection.add(
documents=[doc_text],
metadatas=[{
"room_id": room_id,
"event_type": event_type,
"ts": datetime.now().isoformat(),
**{k: str(v) for k, v in reading.items()}
}],
ids=[f"{room_id}_{datetime.now().timestamp()}"]
)

def recall_similar(self, query: str,
```

```
n_results: int = 3,
filter_room: str = None) -> list:
"""Find semantically similar past readings/incidents."""
where = {}
if filter_room:
where["room_id"] = filter_room

results = self.collection.query(
query_texts=[query],
n_results=n_results,
where=where if where else None
)
return [
{
"text": doc,
"metadata": meta,
"distance": dist,
"similarity": round(1 - dist, 3)
}
for doc, meta, dist in zip(
results["documents"][0],
results["metadatas"][0],
results["distances"][0]
)
]

def recall_anomalies(self, n_results: int = 5) -> list:
"""Find past anomalous readings."""
return self.recall_similar(
"temperature spike HVAC failure high energy anomaly",
n_results=n_results
)



# ── Test ──────────────────────────────────────────────
if __name__ == "__main__":
sm = SemanticMemory()

# Store some readings
sm.store_reading("R301", {
"temp_c": 28.5, "humidity": 75,
"occupancy": True, "kwh": 4.2
}, event_type="HIGH_TEMP_ALERT")

sm.store_reading("R205", {
"temp_c": 22.0, "humidity": 55,
"occupancy": False, "kwh": 1.1
})

# Find similar past incidents
similar = sm.recall_similar(
"room temperature too high HVAC problem"
)
print("Similar past incidents:")
for s in similar:
print(f" [{s['similarity']:.2f}] {s['text'][:70]}")

# Find anomalies
anomalies = sm.recall_anomalies()
print(f"\nPast anomalies found: {len(anomalies)}")
```

Key Insight:	Semantic memory answers "fuzzy" questions like "find rooms similar to this incident" — impossible with Cypher. ChromaDB stores text embeddings; cosine similarity finds the closest matches. Use this alongside Neo4j: vectors for similarity, Cypher for relationships.

■ 1:15–1:30 CHECKPOINT 4: LP Optimizer + ARIMA Forecast

Goal: Implement the LP HVAC optimizer (PuLP) and occupancy forecaster (Prophet). The optimizer reads forecast outputs to set energy-saving HVAC setpoints.

Linear Programming Formulation

Minimize: $\Sigma_i \Sigma_t$ (setpoint_i_t × tariff_t × area_m2_i) | Subject to: temp_min ≤ setpoint_i_t ≤ temp_max rooms i, time t | Σ_i setpoint_i_t × load_factor ≤ peak_capacity (demand limit) | setpoint_i_t = setpoint_i_t-1 + δ, |δ| ≤ 2°C (ramping constraint) | if occupancy_i_t = 0: setpoint_i_t ≥ setpoint_min_unoccupied | Decision variables: setpoint_i_t (HVAC setpoint for room i at time t) | Where: tariff_t = electricity price at time t (TOU pricing)

checkpoint4_optimizer.py

```
# checkpoint4_optimizer.py — LP HVAC + ARIMA Forecasting
import pulp
import numpy as np
import pandas as pd
from prophet import Prophet
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "hotel_mas_2024")
)

# ── Part A: ARIMA/Prophet Occupancy Forecast ──────────
class OccupancyForecaster:
"""Predict room occupancy for next 4 hours using Prophet."""

def __init__(self):
self.models = {} # room_id → fitted Prophet model

def load_history(self, room_id: str) -> pd.DataFrame:
"""Fetch occupancy history from Neo4j."""
with driver.session() as s:
result = s.run("""
MATCH (r:Room {id: $rid})
-[:HAS_DEVICE]->(d:Device {type:'SENSOR'})
-[:RECORDED]->(sr:SensorReading)
RETURN sr.ts AS ds,
CASE WHEN sr.occupancy THEN 1.0
ELSE 0.0 END AS y
ORDER BY sr.ts
LIMIT 1000
""", rid=room_id)
rows = [dict(r) for r in result]
if not rows:
dates = pd.date_range(
end=pd.Timestamp.now(), periods=100, freq='h'
)
return pd.DataFrame({
'ds': dates,
'y': np.random.binomial(1, 0.6, 100).astype(float)
})
return pd.DataFrame(rows)

def fit(self, room_id: str):
"""Fit Prophet model for a room."""
df = self.load_history(room_id)
model = Prophet(
changepoint_prior_scale=0.05,
seasonality_mode='multiplicative',
```

```
daily_seasonality=True,
weekly_seasonality=True
)
model.fit(df)
self.models[room_id] = model

def forecast(self, room_id: str,
hours: int = 4) -> list:
"""Return occupancy probability for next N hours."""
if room_id not in self.models:
self.fit(room_id)
model = self.models[room_id]
future = model.make_future_dataframe(
periods=hours, freq='h'
)
pred = model.predict(future)
probs = np.clip(pred['yhat'].tail(hours).values, 0, 1)
return probs.tolist()


# ── Part B: LP HVAC Optimizer ──────────────────────────
class HVACOptimizer:
"""
Linear Program: minimize energy cost subject to
occupancy, comfort, and demand constraints.
"""
TEMP_MIN_OCC = 20.0 # occupied room min temp
TEMP_MAX_OCC = 26.0 # occupied room max temp
TEMP_MIN_EMPTY = 18.0 # empty room min temp
TEMP_MAX_EMPTY = 30.0 # empty room max (save energy)
TARIFF_PEAK = 5.2 # THB/kWh peak hours (9-21h)
TARIFF_OFF = 2.8 # THB/kWh off-peak
PEAK_HOURS = range(9, 22)

def __init__(self, rooms: list,
forecaster: OccupancyForecaster):
self.rooms = rooms
self.forecaster = forecaster
self.horizon = 4

def optimize(self) -> dict:
"""
Solve LP for optimal HVAC setpoints.
Returns: dict of room_id → recommended_setpoint_C
"""
forecasts = {}
for r in self.rooms:
forecasts[r] = self.forecaster.forecast(
r, self.horizon)

from datetime import datetime
current_hour = datetime.now().hour
tariff = (self.TARIFF_PEAK if current_hour
in self.PEAK_HOURS else self.TARIFF_OFF)

prob = pulp.LpProblem("HVAC_Optimization",
pulp.LpMinimize)

setpoints = {
r: pulp.LpVariable(
f"setpoint_{r}",
lowBound=self.TEMP_MIN_EMPTY,
upBound=self.TEMP_MAX_OCC
```

```
)
for r in self.rooms
}

# Objective: minimize energy proxy
prob += pulp.lpSum(
setpoints[r] * tariff * 0.1
for r in self.rooms
)

# Constraints per room
for r in self.rooms:
occ_prob = forecasts[r][0]
if occ_prob > 0.5:
prob += setpoints[r] >= self.TEMP_MIN_OCC
prob += setpoints[r] <= self.TEMP_MAX_OCC
else:
prob += setpoints[r] >= self.TEMP_MIN_EMPTY
prob += setpoints[r] <= self.TEMP_MAX_EMPTY

prob.solve(pulp.PULP_CBC_CMD(msg=0))

if prob.status != 1:
return {r: 22.0 for r in self.rooms}

return {
r: round(pulp.value(setpoints[r]), 1)
for r in self.rooms
}


# ── Run ──────────────────────────────────────────────
if __name__ == "__main__":
rooms = [f"R{f}{n:02d}"
for f in range(1,4) for n in range(1,6)]
forecaster = OccupancyForecaster()
optimizer = HVACOptimizer(rooms, forecaster)

print("Running LP optimization for 15 rooms...")
setpoints = optimizer.optimize()
for room, sp in list(setpoints.items())[:5]:
print(f" {room}: {sp}°C")

driver.close()
```

Key Insight: The LP solver (PuLP + CBC) finds the optimal HVAC setpoints in <50ms for 200 rooms. Combined with ARIMA/Prophet occupancy forecasting, agents pre-cool occupied rooms and save energy in empty ones — typically 25-35% energy reduction in practice.

■ 1:30–1:45 CHECKPOINT 5: RL Policy + Anomaly Detection

Goal: Train a DQN agent on the hotel HVAC environment using Stable-Baselines3. Also implement Isolation Forest anomaly detection for real-time sensor monitoring.

Q-Learning Bellman Equation

Q(s,a) ← Q(s,a) + α × [r + γ × max_a' Q(s',a') - Q(s,a)] | Where: s = state: (temp_bucket, occupancy, hour_of_day) | a = action: setpoint_delta {-2, -1, 0, +1, +2}°C | r = reward: -1×energy_kwh + 5×(comfort_satisfied) | α = learning rate = 0.1 | γ = discount factor = 0.9 | After 10,000 training episodes: occupied + daytime → setpoint 22-24°C (comfort); empty + peak_hours → setpoint 28-30°C (energy)

checkpoint5_rl_anomaly.py

```
# checkpoint5_rl_anomaly.py — RL + Anomaly Detection
import gymnasium as gym
import numpy as np
from stable_baselines3 import DQN
from stable_baselines3.common.env_util import make_vec_env
from sklearn.ensemble import IsolationForest
import pandas as pd
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "hotel_mas_2024")
)

# ── Part A: Custom Gym Environment ──────────────────────
class HotelHVACEnv(gym.Env):
"""
Custom Gymnasium environment for hotel HVAC control.
State: (temp_c, occupancy, hour_of_day, energy_kwh)
Action: HVAC setpoint delta {-2, -1, 0, +1, +2}
Reward: comfort bonus - energy cost
"""
metadata = {'render_modes': []}

def __init__(self):
super().__init__()
self.observation_space = gym.spaces.Box(
low=np.array([0, 0, 0, 0], dtype=np.float32),
high=np.array([40, 1, 23, 5], dtype=np.float32)
)
self.action_space = gym.spaces.Discrete(5)
self.action_deltas = [-2, -1, 0, +1, +2]
self.setpoint = 22.0
self.temp = 24.0
self.step_count = 0

def reset(self, seed=None, options=None):
super().reset(seed=seed)
self.temp = float(np.random.uniform(20, 30))
self.setpoint = 22.0
self.step_count = 0
hour = float(np.random.randint(0, 24))
occ = float(np.random.binomial(1, 0.6))
state = np.array([self.temp, occ, hour, 1.5],
dtype=np.float32)
return state, {}

def step(self, action):
delta = self.action_deltas[action]
```

```
        self.setpoint = np.clip(self.setpoint + delta, 16, 30)
        self.temp += (self.setpoint - self.temp) * 0.1
        self.temp += np.random.normal(0, 0.2)

        hour = float(np.random.randint(0, 24))
        occupancy = float(np.random.binomial(1, 0.6))
        energy_kwh = max(0, (30 - self.setpoint) * 0.08)

        comfort_ok = 20 <= self.temp <= 26 and occupancy
        reward = (3.0 if comfort_ok else -1.0) - energy_kwh

        state = np.array(
            [self.temp, occupancy, hour, energy_kwh],
            dtype=np.float32
        )
        self.step_count += 1
        done = self.step_count >= 200
        return state, reward, done, False, {}


# ── Part B: Train RL Agent ──────────────────────────────
def train_rl_agent(timesteps: int = 50_000):
    """Train DQN agent on hotel HVAC environment."""
    env = make_vec_env(HotelHVACEnv, n_envs=4)
    model = DQN(
        "MlpPolicy", env,
        learning_rate=1e-3,
        buffer_size=10_000,
        batch_size=64,
        gamma=0.90,
        exploration_fraction=0.2,
        verbose=0
    )
    print(f"Training RL agent ({timesteps:,} steps)...")
    model.learn(total_timesteps=timesteps)
    model.save("/tmp/hotel_hvac_dqn")
    print("Saved to /tmp/hotel_hvac_dqn.zip")
    return model


# ── Part C: Isolation Forest Anomaly Detection ──────────
class SensorAnomalyDetector:
    """
    Detect anomalous sensor readings using Isolation Forest.
    Contamination = 0.05 → flags top 5% as anomalies.
    Isolation Forest path length formula:
    anomaly_score = 2^(-E[h(x)] / c(n))
    Where h(x) is average path length, c(n) is normalisation.
    """
    def __init__(self, contamination: float = 0.05):
        self.model = IsolationForest(
            contamination=contamination,
            random_state=42,
            n_estimators=100
        )
        self.fitted = False

    def fit_from_neo4j(self):
        """Load historical readings from KG for training."""
        with driver.session() as s:
            result = s.run("""
                MATCH (sr:SensorReading)
                RETURN sr.temp_c AS temp_c,
```

```
sr.humidity AS humidity,
sr.kwh AS kwh,
CASE WHEN sr.occupancy
THEN 1 ELSE 0 END AS occupancy
ORDER BY sr.ts DESC LIMIT 5000
""")
rows = [dict(r) for r in result]

if len(rows) < 10:
np.random.seed(42)
rows = [{
"temp_c": np.random.normal(22, 2),
"humidity": np.random.normal(55, 10),
"kwh": np.random.normal(2, 0.5),
"occupancy": np.random.randint(0, 2)
} for _ in range(200)]

df = pd.DataFrame(rows).fillna(0)
X = df[["temp_c","humidity","kwh",
"occupancy"]].values
self.model.fit(X)
self.fitted = True
print(f"Detector trained on {len(X)} readings")

def predict(self, reading: dict) -> dict:
"""Score a single reading."""
if not self.fitted:
self.fit_from_neo4j()
X = [[
reading.get("temp_c", 22),
reading.get("humidity", 55),
reading.get("kwh", 2),
1 if reading.get("occupancy") else 0
]]
pred = self.model.predict(X)[0]
score = self.model.score_samples(X)[0]
return {
"anomaly": pred == -1,
"anomaly_score": round(float(score), 4),
"interpretation": (
"ANOMALOUS" if pred == -1 else "NORMAL"
)
}



# ── Test ──────────────────────────────────────────────
if __name__ == "__main__":
detector = SensorAnomalyDetector()
detector.fit_from_neo4j()

test_readings = [
{"temp_c": 22.0, "humidity": 55,
"kwh": 2.0, "occupancy": True},
{"temp_c": 38.5, "humidity": 95,
"kwh": 5.8, "occupancy": False},
]
for reading in test_readings:
result = detector.predict(reading)
print(f" {reading['temp_c']}°C → "
f"{result['interpretation']} "
f"(score: {result['anomaly_score']})")

model = train_rl_agent(timesteps=10_000)
```

```
driver.close()
```

Key Insight: The Isolation Forest anomaly score is inversely proportional to the average path length in isolation trees. Normal points are hard to isolate (long path) → high score. Anomalies are easy to isolate (short path) → low score. Contamination=0.05 flags the 5% most unusual readings.

■ 1:45–2:00 CHECKPOINT 6: Full MAS + GraphRAG Demo

Goal: Wire all 4 memory layers into a 5-agent CrewAI system. Each agent reads/writes the appropriate memory layers. Run the full hotel monitoring loop end-to-end.

checkpoint6_full_mas.py — Part 1: KG Memory

```
# checkpoint6_full_mas.py — Complete Smart Hotel MAS
# Part 1: Knowledge Graph Memory (L4)
from neo4j import GraphDatabase
import json

driver = GraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "hotel_mas_2024")
)

class AgentMemoryBase:
"""Base class providing Neo4j KG access."""
def __init__(self, agent_id: str):
self.agent_id = agent_id

def cypher(self, query: str, **params) -> list:
with driver.session() as s:
result = s.run(query, **params)
return [dict(r) for r in result]

def store_event(self, event_type: str,
data: dict,
entities: list = None):
"""Store an agent event in the KG."""
self.cypher("""
CREATE (e:Event {
type: $etype,
agent_id: $aid,
data: $data,
ts: datetime()
})
""",
etype=event_type,
aid=self.agent_id,
data=json.dumps(data)
)

def recall_recent(self, hours: int = 8,
limit: int = 20) -> list:
return self.cypher("""
MATCH (e:Event {agent_id: $aid})
WHERE e.ts > datetime() - duration({hours: $h})
RETURN e.type AS type, e.data AS data,
toString(e.ts) AS ts
ORDER BY e.ts DESC LIMIT $lim
""", aid=self.agent_id, h=hours, lim=limit)

def close(self):
driver.close()


class HotelKGMemory(AgentMemoryBase):
"""Extended KG memory for hotel domain."""

def store_sensor_batch(self, room_id, reading, anomaly):
self.store_event(
```

```
event_type="sensor_reading",
data={**reading, "anomaly": anomaly},
entities=[
("Room", room_id),
("Device", f"SENSOR_{room_id}")
]
)
if anomaly:
self.cypher("""
MERGE (r:Room {id: $rid})
CREATE (a:Alert {
type: 'SENSOR_ANOMALY',
severity: 'HIGH',
ts: datetime(),
data: $data
})
CREATE (r)<-[:AFFECTS]-(a)
""", rid=room_id, data=json.dumps(reading))

def store_hvac_action(self, room_id, setpoint,
energy_saved_kwh):
self.store_event(
event_type="hvac_optimization",
data={"setpoint": setpoint,
"energy_saved": energy_saved_kwh},
entities=[("Room", room_id),
("Device", f"HVAC_{room_id}")]
)

def query_floor_summary(self, floor: int) -> list:
return self.cypher("""
MATCH (r:Room)-[:HAS_DEVICE]->
(d:Device {type:'SENSOR'})
-[:RECORDED]->(sr:SensorReading)
WHERE r.floor = $floor
AND sr.ts > datetime() - duration({hours:1})
RETURN r.id AS room,
avg(sr.temp_c) AS avg_temp,
avg(sr.kwh) AS avg_kwh,
count(sr) AS readings
ORDER BY avg_kwh DESC
""", floor=floor)

def query_active_alerts(self) -> list:
return self.cypher("""
MATCH (a:Alert)-[:AFFECTS]->(r:Room)
WHERE a.ts > datetime() - duration({hours:4})
RETURN r.id AS room,
a.type AS alert_type,
a.severity AS severity,
toString(a.ts) AS when
ORDER BY a.ts DESC LIMIT 10
""")
```

checkpoint6_full_mas.py — Part 2: CrewAI Crew

```
# Part 2: Full CrewAI Multi-Agent System
from crewai import Agent, Task, Crew, Process
from crewai_tools import BaseTool
import random, json

# Shared memory instances (all agents share these)
kg_mem = HotelKGMemory("HotelMAS")
```

```
class SensorReadTool(BaseTool):
name: str = "read_sensors"
description: str = (
"Read IoT sensor data for a batch of rooms. "
"Returns temp, humidity, energy, occupancy."
)
def _run(self, room_ids: str) -> str:
rooms = room_ids.split(',')
readings = {}
for r in rooms:
readings[r.strip()] = {
"temp_c": round(20 + random.gauss(4,2), 1),
"humidity": round(40 + random.random()*30, 1),
"kwh": round(1 + random.random()*3, 2),
"occupancy": random.choice([True,False])
}
return json.dumps(readings)


class OptimizeTool(BaseTool):
name: str = "optimize_hvac"
description: str = (
"Run LP optimization to get recommended HVAC "
"setpoints for a list of rooms."
)
def _run(self, room_ids: str) -> str:
rooms = [r.strip() for r in room_ids.split(',')]
setpoints = {r: round(20 + random.random()*6, 1)
for r in rooms}
for room, sp in setpoints.items():
kg_mem.store_hvac_action(room, sp, 0.3)
return json.dumps(setpoints)


class GraphRAGTool(BaseTool):
name: str = "query_knowledge_graph"
description: str = (
"Query the hotel knowledge graph for status, "
"history, and patterns using natural language."
)
def _run(self, query: str) -> str:
if "alert" in query.lower():
result = kg_mem.query_active_alerts()
elif "floor" in query.lower():
floor = int(''.join(
filter(str.isdigit, query)) or '3')
result = kg_mem.query_floor_summary(floor)
else:
result = kg_mem.recall_recent(limit=10)
return json.dumps(result, default=str)[:500]


# 5 Specialized Agents
sensor_agent = Agent(
role="Hotel Sensor Monitor",
goal="Read all room sensors, detect anomalies",
backstory="IoT specialist monitoring 200 rooms 24/7",
tools=[SensorReadTool()],
llm="claude-3-5-sonnet-20241022",
verbose=False
)
```

```
energy_agent = Agent(
role="Energy Optimization Engineer",
goal="Minimize energy while maximizing guest comfort",
backstory="Expert using LP and RL for HVAC control",
tools=[OptimizeTool()],
llm="claude-3-5-sonnet-20241022",
verbose=False
)
memory_agent = Agent(
role="Hotel Memory Manager",
goal="Store all events to the 4-layer memory system",
backstory="Data steward recording all agent actions",
tools=[GraphRAGTool()],
llm="claude-3-5-sonnet-20241022",
verbose=False
)
alert_agent = Agent(
role="Hotel Alert Coordinator",
goal="Detect anomalies, create maintenance jobs",
backstory="Facilities manager with IoT expertise",
tools=[GraphRAGTool()],
llm="claude-3-5-sonnet-20241022",
verbose=False
)
report_agent = Agent(
role="Hotel Intelligence Reporter",
goal="Generate natural language hotel status reports",
backstory="Hotel ops analyst using GraphRAG for insights",
tools=[GraphRAGTool()],
llm="claude-3-5-sonnet-20241022",
verbose=False
)

# Tasks
t1 = Task(
description=(
"Read sensors for rooms R101,R102,R103,R201,R202. "
"Report anomalous rooms."
),
expected_output="Dict: room→{reading, anomaly result}",
agent=sensor_agent
)
t2 = Task(
description=(
"For rooms R101,R102,R103,R201,R202, run LP "
"optimization to get optimal HVAC setpoints."
),
expected_output="Dict: room→recommended_setpoint_C",
agent=energy_agent
)
t3 = Task(
description=(
"Query the knowledge graph: list all active alerts "
"and energy optimization actions taken today."
),
expected_output="Summary of alerts and optimizations",
agent=memory_agent
)
t4 = Task(
description=(
"Generate a 2-paragraph hotel operations report: "
"current status, anomalies, energy savings, "
"and recommended actions for next hour."
```

```
),
expected_output="Professional hotel ops report text",
agent=report_agent
)

hotel_crew = Crew(
agents=[sensor_agent, energy_agent,
memory_agent, report_agent],
tasks=[t1, t2, t3, t4],
process=Process.sequential,
verbose=False
)

if __name__ == "__main__":
print("=" * 50)
print("Smart Hotel MAS — Full System Test")
print("=" * 50)
result = hotel_crew.kickoff()
print("\nFinal Report:")
print(result)
kg_mem.close()
driver.close()
```

Key Insight: The full MAS now uses all 4 memory layers: L1 (working dict for current readings), L2 (SQLite for run history), L3 (ChromaDB for similar incident recall), L4 (Neo4j KG for structured entity relationships). Each layer serves a different retrieval need — together they give agents complete situational awareness.

Mathematical Models — Deep Dive

Formal equations for LP, ARIMA, Q-learning, and Isolation Forest

Linear Programming — Full Formulation

LP Formulation — HVAC Energy Optimization

Decision Variables: x_{r,t} = HVAC setpoint for room r at time t (°C) Objective: Minimize Σ_r Σ_t c_t × k_r × max(0, x_r,t - T_ambient) Where: c_t = electricity tariff at time t (THB/kWh) k_r = room heat transfer coefficient (kW/°C) Subject to: (Comfort) 20 ≤ x_{r,t} ≤ 26 ∀ r ∈ occupied_rooms, ∀ t (Empty) 18 ≤ x_{r,t} ≤ 30 ∀ r ∈ empty_rooms, ∀ t (Ramp) |x_{r,t} - x_{r,t-1}| ≤ 2 ∀ r, t>0 (°C/15min) (Peak demand) Σ_r load_r(x_{r,t}) ≤ peak_capacity ∀ t ∈ peak_hours (Non-negativity) x_{r,t} ≥ 0 Solution: PuLP CBC solver, typically finds optimum in <100ms for 200 rooms.

ARIMA/Prophet Occupancy Forecast

ARIMA(p,d,q) — Occupancy Time Series

ARIMA(p,d,q) combines: AR(p): y_t = c + φ_1 y_{t-1} + ... + φ_p y_{t-p} + ε_t I(d): d-th differencing for stationarity MA(q): y_t = μ + ε_t + θ_1 ε_{t-1} + ... + θ_q ε_{t-q} Prophet decomposes: y(t) = trend(t) + seasonality(t) + holidays(t) + ε_t trend(t) = k + a(t)ᵀ δ + (m + a(t)ᵀ γ) — piecewise linear changepoints seasonality: Fourier series s(t) = Σ(a_n cos(2πnt/P) + b_n sin(2πnt/P)) For hotel occupancy: P=24 (daily) and P=168 (weekly) are dominant periods. Confidence interval: ŷ_{t+h} ± 1.96 × σ_h where σ_h grows with horizon h.

Q-Learning — Bellman Equation

Q-Learning — Bellman Optimality

Bellman Optimality Equation: Q*(s,a) = R(s,a) + γ × Σ_{s'} P(s'|s,a) × max_{a'} Q*(s',a') TD Update rule: Q(s,a) ← Q(s,a) + α[R(s,a) + γ max_{a'} Q(s',a') - Q(s,a)] Hotel HVAC State space: S = {temp_bucket × occupancy × hour} temp_bucket: {<20, 20-22, 22-24, 24-26, >26} = 5 buckets occupancy: {0, 1} = 2 states hour_of_day: {0..23} = 24 states |S| = 5 × 2 × 24 = 240 states Action space: A = {-2, -1, 0, +1, +2}°C delta = 5 actions Total Q-table size: 240 × 5 = 1,200 entries Reward function: R(s,a) = comfort_bonus - energy_cost comfort_bonus = +5 if (20 ≤ temp ≤ 26 AND occupancy=1) energy_cost = 0.1 × |setpoint - 30| (proxy for kWh)

Isolation Forest — Anomaly Score

Isolation Forest — Anomaly Score Formula

For a dataset of n samples, the anomaly score for point x is: s(x,n) = 2^(-E[h(x)] / c(n)) Where: h(x) = path length of x in isolation tree E[h(x)] = average path length across all trees c(n) = average path length of unsuccessful BST search = 2H(n-1) - 2(n-1)/n H(i) = harmonic number ≈ ln(i) + 0.5772 (Euler-Mascheroni constant) Interpretation: s(x,n) ≈ 1 → anomaly (short path, isolated early) s(x,n) ≈ 0.5 → normal (average path length) s(x,n) ≈ 0 → far from anomaly For hotel sensors: contamination=0.05 means the top 5% of readings by anomaly score are flagged as anomalies. This corresponds to: temp_c > 30°C OR temp_c < 15°C OR kwh > 5 kWh (typically)

Model Performance Comparison

Model	Training Time	Inference Time	Accuracy	Best For
PuLP LP	N/A (no training)	<100ms (200 rooms)	Optimal (if constraints correct)	Deterministic optimization with known constraints
Prophet ARIMA	5-30s per room	<1s	MAPE ~15-25% for 4h horizon	Seasonal time series with trend + holiday effects
DQN (SB3)	10-60min GPU	<1ms	Converges to near-optimal policy	Unknown dynamics, long-horizon optimization

Model	Training Time	Inference Time	Accuracy	Best For
Isolation Forest	0.5-2s	<1ms per reading	AUC-ROC 0.85-0.95 typical	Unsupervised anomaly detection, no labeled data needed

Memory Layer Performance Summary

Layer	Technology	Write Time	Read Time	Capacity	Persistence
L1 Working	Python dict	<0.01ms	<0.01ms	RAM-bound (~100MB)	Process lifetime
L2 Episodic	SQLite	0.1-1ms	1-10ms (indexed)	Disk-bound (GB)	Permanent
L3 Semantic	ChromaDB	10-100ms (embed)	50-200ms (ANN)	Millions of vectors	Permanent
L4 KG	Neo4j	1-10ms	10-200ms (Cypher)	Billions of nodes/edges	Permanent

Agent Design Patterns

ReAct loop, tool calling, memory injection, and error handling

The ReAct Loop: Reasoning + Acting

Every agent in this system follows the ReAct (Reasoning + Acting) pattern. The LLM reasons about what to do, calls a tool, observes the result, and reasons again. Memory is injected at each step.

ReAct Loop Trace

```
ReAct Loop for SensorAgent:

[1] THOUGHT: "I need to read sensors for the current batch"
↓
[2] ACTION: read_sensors(room_ids="R101,R102,R103")
↓
[3] OBSERVATION: {"R101": {"temp_c": 27.5, "anomaly": true}, ...}
↓
[4] THOUGHT: "R101 has high temp. I should check anomaly details"
↓
[5] ACTION: check_anomaly(room_id="R101")
↓
[6] OBSERVATION: {"anomaly": true, "score": -0.42, "interp": "ANOMALOUS"}
↓
[7] THOUGHT: "Confirmed anomaly. Store to memory layers and alert."
↓
[8] FINAL ANSWER: {"anomalous_rooms": ["R101"], "details": {...}}

Memory injected at each step:
L1 working_mem.summary() → "Working Memory: {batch=['R101','R102'], ...}"
L2 episodic_mem.recall_recent(hours=1) → last 5 events
These appear in the system prompt for every LLM call.
```

Memory Injection Pattern

memory_injection.py

```
# memory_injection.py — Inject memory into agent context

def build_agent_context(working_mem: WorkingMemory,
episodic_mem: EpisodicMemory,
kg_mem: HotelKGMemory,
agent_id: str) -> str:
"""
Build enriched context string for LLM prompt.
Called before every agent invocation.
"""
context_parts = []

# L1: Current working state
wm_summary = working_mem.summary()
context_parts.append(f"[L1 Working Memory]\n{wm_summary}")

# L2: Recent episodic history (last hour)
recent_eps = episodic_mem.recall_recent(hours=1, limit=5)
if recent_eps:
ep_lines = [
f" {ep['ts'][:19]} {ep['agent']}: {ep['type']}"
for ep in recent_eps
]
```

```
context_parts.append(
"[L2 Episodic Memory - Last Hour]\n" +
"\n".join(ep_lines)
)

# L4: Active alerts from KG
alerts = kg_mem.query_active_alerts()
if alerts:
alert_lines = [
f" ALERT {a['alert_type']} in {a['room']} "
f"(severity: {a['severity']})"
for a in alerts[:3]
]
context_parts.append(
"[L4 KG - Active Alerts]\n" +
"\n".join(alert_lines)
)

return "\n\n".join(context_parts)


def inject_context_into_agent(agent: 'Agent',
context: str) -> 'Agent':
"""
Prepend memory context to agent backstory.
CrewAI includes backstory in every LLM call.
"""
agent.backstory = (
f"{agent.backstory}\n\n"
f"--- Current Context ---\n{context}"
)
return agent


# Usage in main loop
def run_agent_cycle(working_mem, episodic_mem, kg_mem):
context = build_agent_context(
working_mem, episodic_mem, kg_mem, "SensorAgent"
)
# Context is ~200-500 tokens — well within LLM limits
print(f"Context tokens: ~{len(context.split())}")
return context
```

Key Insight: Memory injection converts stored data into natural language that the LLM can reason about. Keep injected context <500 tokens to avoid degrading LLM performance. Use L1 for current state, L2 for recent history, L4 for structured alerts.

Error Handling and Retry Logic

error_handling.py

```
# error_handling.py — Robust agent error handling

import functools
import time
from typing import Callable, Any

def with_retry(max_retries: int = 3,
backoff: float = 1.0,
exceptions: tuple = (Exception,)):
"""Decorator: retry tool calls on transient failures."""
def decorator(fn: Callable) -> Callable:
@functools.wraps(fn)
```

```
def wrapper(*args, **kwargs) -> Any:
last_error = None
for attempt in range(max_retries):
try:
return fn(*args, **kwargs)
except exceptions as e:
last_error = e
wait = backoff * (2 ** attempt)
print(f"Attempt {attempt+1} failed: {e}. "
f"Retrying in {wait:.1f}s...")
time.sleep(wait)
raise last_error
return wrapper
return decorator


class SafeNeo4jTool(BaseTool):
"""Tool with automatic Neo4j reconnection."""
name: str = "safe_kg_query"
description: str = "Query KG with automatic retry"

@with_retry(max_retries=3,
exceptions=(Exception,))
def _run(self, query: str) -> str:
try:
result = kg_mem.cypher(query)
return json.dumps(result, default=str)
except Exception as e:
# Fallback to episodic memory
recent = episodic_mem.recall_recent(hours=4)
return (f"KG unavailable ({e}). "
f"Episodic fallback: {len(recent)} events")


class GracefulDegradation:
"""
Fallback strategy when memory layers fail.
Hierarchy: L4(KG) → L3(Chroma) → L2(SQLite) → L1(dict)
"""

def query_with_fallback(self,
query: str,
kg: HotelKGMemory,
semantic: SemanticMemory,
episodic: EpisodicMemory,
working: WorkingMemory) -> dict:
# Try L4 first
try:
result = kg.cypher(
"MATCH (e:Event) RETURN e LIMIT 5")
return {"source": "L4_KG", "data": result}
except Exception:
pass

# Fall back to L3 semantic
try:
result = semantic.recall_similar(query, n_results=3)
return {"source": "L3_semantic", "data": result}
except Exception:
pass

# Fall back to L2 episodic
try:
```

```
result = episodic.recall_recent(hours=8, limit=10)
return {"source": "L2_episodic", "data": result}
except Exception:
pass

# Last resort: L1 working memory
return {
"source": "L1_working",
"data": working.get_all()
}
```

Tool Design Best Practices

Pattern	Problem Solved	Implementation	Example
Atomic Tools	Tools that do one thing well are easier to compose	One tool = one responsibility; avoid side effects in description	read_sensor vs read_sensor_and_optimize (bad)
Typed Outputs	LLM needs consistent output format to parse results	Always return JSON string; include status field	{"status": "ok", "rooms": [...], "anomalies": 0}
Bounded Results	Prevent tool output overflowing LLM context window	Add LIMIT to all KG queries; truncate strings to 500 chars	return json.dumps(result)[:500]
Memory Write-Through	Keep all layers consistent on every write	Tool writes to L1, L2, and L4 in sequence; L3 asynchronously	set L1 → insert L2 → CREATE L4 node
Idempotent MERGE	Prevent duplicate nodes from retry logic	Always use MERGE not CREATE for entity nodes	MERGE (r:Room {id: $rid}) SET r += $props

GraphRAG Deep Dive

Natural language → Cypher → knowledge graph context → LLM response

GraphRAG (Graph Retrieval Augmented Generation) combines a knowledge graph with an LLM. The LLM generates Cypher queries, executes them against Neo4j, and uses the results as grounding context for its final answer.

GraphRAG Pipeline Architecture

GraphRAG Pipeline Trace

```
GraphRAG Pipeline for Hotel Intelligence:

User Question: "Which floor had the most energy use last night?"
↓
[Step 1: Schema Injection]
System prompt includes KG schema:
"Nodes: Room(id,floor), Device(type), SensorReading(ts,kwh)
Rels: (:Room)-[:HAS_DEVICE]->(:Device)-[:RECORDED]->(SensorReading)"
↓
[Step 2: Cypher Generation (Claude)]
Claude generates:
MATCH (r:Room)-[:HAS_DEVICE]->(d:Device {type:'SENSOR'})
-[:RECORDED]->(sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours: 12})
RETURN r.floor AS floor, sum(sr.kwh) AS total_kwh
ORDER BY total_kwh DESC LIMIT 1
↓
[Step 3: Cypher Execution (Neo4j)]
Result: [{"floor": 3, "total_kwh": 847.2}]
↓
[Step 4: Context Assembly]
"Based on Neo4j query result: Floor 3 consumed 847.2 kWh
last night (highest of all 5 floors)"
↓
[Step 5: LLM Final Answer]
"Floor 3 had the highest energy consumption last night at
847.2 kWh. This is 23% above the building average of
689 kWh. Recommend HVAC audit for Floor 3 rooms."

Total latency: 150-400ms (Cypher: 50ms + LLM: 100-350ms)
```

GraphRAG Implementation

graphrag.py

```
# graphrag.py — Hotel GraphRAG with LangChain + Claude

from langchain_anthropic import ChatAnthropic
from langchain_community.graphs import Neo4jGraph
from langchain.chains import GraphCypherQAChain
from langchain.prompts import PromptTemplate

HOTEL_SCHEMA = """
Node Labels and Properties:
- Room: {id, floor, type, status, rate_thb}
- Device: {id, type, room_id, status}
- SensorReading: {ts, temp_c, humidity, kwh, occupancy}
- Alert: {type, severity, ts}
- MaintenanceJob: {type, priority, status}
```

```
- Staff: {id, name, role, shift, available}
- Event: {type, agent_id, ts, data}

Relationship Types:
- (:Room)-[:HAS_DEVICE]->(:Device)
- (:Device)-[:RECORDED]->(:SensorReading)
- (:Alert)-[:AFFECTS]->(:Room)
- (:MaintenanceJob)-[:ASSIGNED_TO]->(:Staff)
- (:Agent)-[:PERFORMED]->(:Event)
"""

CYPHER_PROMPT = PromptTemplate(
input_variables=["schema", "question"],
template="""
You are an expert Neo4j Cypher query generator for a hotel MAS.
Schema: {schema}

Rules:
1. Always use LIMIT to cap results (max 20)
2. Convert Neo4j datetime to string with toString()
3. Use avg(), sum(), count() for aggregations
4. Use datetime() - duration({hours: N}) for time filters
5. Return meaningful column aliases (e.g., AS room, AS temp_c)

Question: {question}

Cypher query (no explanation, just the query):"""
)

class HotelGraphRAG:
def __init__(self):
self.graph = Neo4jGraph(
url="bolt://localhost:7687",
username="neo4j",
password="hotel_mas_2024"
)
self.llm = ChatAnthropic(
model="claude-3-5-sonnet-20241022",
temperature=0
)
self.chain = GraphCypherQAChain.from_llm(
llm=self.llm,
graph=self.graph,
cypher_prompt=CYPHER_PROMPT,
verbose=True,
return_intermediate_steps=True,
top_k=10
)

def query(self, question: str) -> dict:
"""Execute GraphRAG query, return answer + Cypher."""
result = self.chain({"query": question,
"schema": HOTEL_SCHEMA})
return {
"answer": result["result"],
"cypher": result["intermediate_steps"][0].get(
"query", ""),
"kg_result": result["intermediate_steps"][1].get(
"context", [])
}


# ── Demonstration queries ──────────────────────────────
```

```
if __name__ == "__main__":
rag = HotelGraphRAG()

questions = [
"Which rooms had temperature above 28°C today?",
"What is the average energy consumption per floor?",
"Which staff members are available for HVAC repairs?",
"How many alerts were triggered in the last 4 hours?",
]

for q in questions:
print(f"\nQ: {q}")
result = rag.query(q)
print(f"Cypher: {result['cypher'][:80]}...")
print(f"Answer: {result['answer'][:150]}")
```

GraphRAG Query Quality: Common Patterns

Question Type	Natural Language Pattern	Expected Cypher Pattern	Notes
Aggregation	"What is the average temperature on floor 3?"	MATCH ... WHERE r.floor=3 RETURN avg(sr.temp_c)	Always filter by time range too
Existence	"Are there any HIGH severity alerts right now?"	MATCH (a:Alert) WHERE a.severity="HIGH" RETURN count(a)	count() not exists() for LLM compatibility
Path traversal	"Which devices caused the most alerts?"	MATCH (a:Alert)-[:CAUSED_BY]->(d) RETURN d, count(a)	Multi-hop path queries work well
Top-N	"Which 5 rooms use the most energy?"	MATCH ... RETURN r, sum(kwh) ORDER BY sum DESC LIMIT 5	LIMIT essential to cap LLM context
Temporal	"What happened in Room 205 yesterday?"	MATCH ... WHERE sr.ts > datetime()-duration({hours:24})	Always use Neo4j datetime() not ISO string

GraphRAG Tip: Schema Injection

The most important factor in GraphRAG quality is schema injection. If the LLM does not know about :SensorReading nodes and -[:RECORDED]-> edges, it cannot generate correct Cypher. Always inject the full schema (node labels + relationship types + key properties) into the system prompt. Keep it <200 tokens.

Production Deployment Guide

From workshop prototype to hotel production system

Production Architecture Overview

Production Architecture

```
Production Deployment (Cloud):


┌─────────────────────────────────────────────────┐
│ Frontend: Streamlit Dashboard │
│ (Hotel Manager Chat + Real-time Metrics) │
└──────────────────┬──────────────────────────────┘
│ HTTPS
┌──────────────────▼──────────────────────────────┐
│ API Layer: FastAPI (Render.com) │
│ POST /run_cycle GET /alerts GET /report │
└──────┬──────────┬──────────┬────────────────────┘
│ │ │
┌──────▼─┐ ┌──────▼─┐ ┌──────▼──┐
│ Neo4j │ │ChromaDB │ │ Redis │
│ Aura │ │ Cloud │ │ Cloud │
│(managed)│ │(managed)│ │(managed) │
└────────┘ └────────┘ └─────────┘
│
┌──────▼──────────────────────────────────────────┐
│ MQTT Broker (Eclipse Mosquitto / HiveMQ) │
│ Topic: hotel/floor/{n}/room/{id}/sensor │
└──────────────────────────────────────────────────┘
│
┌──────▼──────────────────────────────────────────┐
│ IoT Devices: Raspberry Pi + ESP32 sensors │
│ 200 rooms × (HVAC + sensor + occupancy) │
└──────────────────────────────────────────────────┘


Cost estimate (Thai hotel, 200 rooms):
Neo4j Aura Professional: $65/month
ChromaDB Cloud: $25/month
Redis Cloud: $10/month
Render API (2 instances): $28/month
MQTT (HiveMQ): $19/month
Claude API: ~$50-100/month (est.)
Total: ~$200-250/month
```

FastAPI Backend

api/main.py

```python
# api/main.py — FastAPI production backend

from fastapi import FastAPI, BackgroundTasks
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from datetime import datetime
import asyncio

app = FastAPI(title="Smart Hotel MAS API", version="1.0")
app.add_middleware(CORSMiddleware,
allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])
```

```
# Shared instances (module-level singletons)
from memory.working import WorkingMemory
from memory.episodic import EpisodicMemory
from memory.semantic import SemanticMemory
from memory.knowledge_graph import HotelKGMemory

wm = WorkingMemory()
em = EpisodicMemory("hotel_prod.db")
sm = SemanticMemory(host="chroma.cloud.example.com")
kg = HotelKGMemory("ProductionMAS")


class SensorBatch(BaseModel):
room_ids: list[str]
readings: dict # room_id → {temp_c, humidity, kwh, occ}


@app.post("/api/v1/sensor_batch")
async def process_sensor_batch(
batch: SensorBatch,
background: BackgroundTasks
):
"""Process a batch of IoT sensor readings."""
results = {}
for room_id in batch.room_ids:
reading = batch.readings.get(room_id, {})
# L1: store in working memory
wm.set(f"latest_{room_id}", reading)
results[room_id] = {"stored": True}

# Background: write to L2, L3, L4
background.add_task(
persist_to_memory_layers, batch.room_ids,
batch.readings, em, sm, kg
)

return {"status": "ok", "rooms": len(batch.room_ids),
"ts": datetime.now().isoformat()}


@app.get("/api/v1/alerts")
async def get_active_alerts(hours: int = 4):
"""Get active alerts from Neo4j KG."""
alerts = kg.query_active_alerts()
return {"alerts": alerts, "count": len(alerts)}


@app.get("/api/v1/floor/{floor_num}/summary")
async def floor_summary(floor_num: int):
"""Get floor energy and temperature summary."""
summary = kg.query_floor_summary(floor_num)
return {"floor": floor_num, "rooms": summary}


@app.post("/api/v1/optimize")
async def run_optimization(room_ids: list[str]):
"""Run LP HVAC optimization for a set of rooms."""
from models.hvac_optimizer import HVACOptimizer
from models.occupancy_forecaster import OccupancyForecaster
forecaster = OccupancyForecaster()
optimizer = HVACOptimizer(room_ids, forecaster)
setpoints = optimizer.optimize()
```

```
return {"setpoints": setpoints, "rooms": len(room_ids)}


@app.get("/api/v1/report")
async def generate_report():
"""Generate hotel intelligence report via GraphRAG."""
from graphrag import HotelGraphRAG
rag = HotelGraphRAG()
result = rag.query(
"Generate a hotel operations summary: total energy, "
"active alerts, occupancy rate, top issues this hour."
)
return {"report": result["answer"],
"cypher": result["cypher"]}


async def persist_to_memory_layers(room_ids, readings,
em, sm, kg):
"""Background task: write to L2, L3, L4."""
for room_id in room_ids:
reading = readings.get(room_id, {})
em.store("SensorAgent", "batch_read",
{"room": room_id, **reading})
sm.store_reading(room_id, reading)
# L4 KG write (lightweight: no anomaly check here)
kg.store_event("sensor_batch",
{"room": room_id, **reading})
```

MQTT Real-time Sensor Integration

mqtt_bridge.py

```
# mqtt_bridge.py — Real-time MQTT → MAS bridge

import paho.mqtt.client as mqtt
import json
import requests
from datetime import datetime

MQTT_BROKER = "localhost"
API_URL = "http://localhost:8000/api/v1"
BATCH_SIZE = 10 # batch readings before API call
TOPIC_PATTERN = "hotel/floor/+/room/+/sensor"

pending_batch = {} # room_id → reading

def on_message(client, userdata, msg):
global pending_batch

# Parse topic: hotel/floor/3/room/R301/sensor
parts = msg.topic.split('/')
floor = int(parts[2])
room_id = parts[4]

# Parse payload
try:
payload = json.loads(msg.payload.decode())
payload['ts'] = datetime.now().isoformat()
pending_batch[room_id] = payload
except json.JSONDecodeError:
return

# Send batch when full
```

```
if len(pending_batch) >= BATCH_SIZE:
flush_batch()


def flush_batch():
global pending_batch
if not pending_batch:
return

room_ids = list(pending_batch.keys())
try:
resp = requests.post(
f"{API_URL}/sensor_batch",
json={
"room_ids": room_ids,
"readings": pending_batch
},
timeout=5
)
print(f"Sent batch: {len(room_ids)} rooms, "
f"status={resp.status_code}")
except requests.RequestException as e:
print(f"API error: {e}")
finally:
pending_batch = {}


def main():
client = mqtt.Client()
client.on_message = on_message
client.connect(MQTT_BROKER, 1883, 60)
client.subscribe(TOPIC_PATTERN)
print(f"Subscribed to {TOPIC_PATTERN}")
client.loop_forever()

if __name__ == "__main__":
main()
```

Key Insight: MQTT is the standard IoT protocol for hotel sensor networks. Eclipse Mosquitto is free and runs on a Raspberry Pi. The MQTT bridge batches readings (10 rooms) before calling the MAS API, reducing database round-trips by 10x and keeping Neo4j write latency below 50ms.

Streamlit Hotel Dashboard

dashboard.py

```
# dashboard.py — Streamlit hotel intelligence dashboard

import streamlit as st
import requests
import pandas as pd
import json

API_URL = "http://localhost:8000/api/v1"

st.set_page_config(
page_title="Smart Hotel MAS Dashboard",
page_icon="🏨", layout="wide"
)

st.title("Smart Hotel MAS — Operations Dashboard")

# Sidebar: GraphRAG chat
```

```
st.sidebar.header("GraphRAG Chat")
question = st.sidebar.text_input(
"Ask the hotel AI:",
placeholder="Which rooms have high energy today?"
)
if st.sidebar.button("Ask"):
with st.sidebar.spinner("Querying KG..."):
resp = requests.post(f"{API_URL}/report",
json={"question": question})
result = resp.json()
st.sidebar.success(result["answer"])
st.sidebar.code(result["cypher"], language="cypher")

# Main: Metrics columns
col1, col2, col3, col4 = st.columns(4)

alerts = requests.get(f"{API_URL}/alerts?hours=1").json()
col1.metric("Active Alerts (1h)", alerts["count"],
delta=None, delta_color="inverse")

col2.metric("Rooms Monitored", "200", "200 online")
col3.metric("Avg Temp (°C)", "23.4", "+0.2°C")
col4.metric("Energy Today (kWh)", "4,847", "-12% vs yesterday")

# Floor energy chart
st.subheader("Floor Energy Summary")
floor_data = []
for f in range(1, 6):
summary = requests.get(
f"{API_URL}/floor/{f}/summary").json()
floor_data.extend([
{"floor": f"Floor {f}",
"room": r["room"],
"kwh": float(r.get("avg_kwh", 0) or 0)}
for r in summary["rooms"]
])

if floor_data:
df = pd.DataFrame(floor_data)
floor_totals = df.groupby("floor")["kwh"].sum().reset_index()
st.bar_chart(floor_totals.set_index("floor"))

# Active alerts table
st.subheader("Active Alerts")
if alerts["alerts"]:
st.dataframe(pd.DataFrame(alerts["alerts"]))
else:
st.success("No active alerts — all systems normal")
```

Workshop Exercises

Hands-on challenges to deepen understanding

Exercise Set A: Memory Layers

Complete these exercises independently during or after the workshop. Each exercise builds on the checkpoint code.

Exercise A: Memory Layer Challenges

```
# Exercise A1: Extend WorkingMemory with LRU eviction
# Problem: WorkingMemory grows unbounded if not cleared.
# Task: Add an LRU (Least Recently Used) cache with max_size.

from collections import OrderedDict

class LRUWorkingMemory:
def __init__(self, max_size: int = 100):
self._cache = OrderedDict()
self._max = max_size

def set(self, key: str, value) -> None:
# TODO: Implement LRU eviction
# Hint: use self._cache.move_to_end(key)
# and self._cache.popitem(last=False)
pass

def get(self, key: str, default=None):
# TODO: Move accessed key to end (most recent)
pass

# Exercise A2: Add EpisodicMemory search by agent_id
class EpisodicMemoryV2(EpisodicMemory):
def recall_by_agent(self, agent_id: str,
hours: int = 24) -> list:
# TODO: Query SQLite for events by a specific agent
# within the last N hours
pass

# Exercise A3: Implement SemanticMemory deduplication
# Problem: If same room reading stored twice, Chroma gets
# duplicate vectors. Fix using content-hash IDs.
import hashlib

def generate_reading_id(room_id: str, reading: dict) -> str:
# TODO: Return deterministic ID based on content
# Hint: hash room_id + str(reading) → MD5[:16]
pass
```

Exercise Set B: Knowledge Graph

Exercise B: Cypher Query Challenges

```
# Exercise B1: Write a Cypher query for alert hotspots
# Find rooms that had >2 HIGH severity alerts in the last 7 days
# Expected: room_id, floor, alert_count, most_recent_alert

# TODO: Write the Cypher query
HOTSPOT_QUERY = """
```

```
"""

# Exercise B2: Find under-maintained devices
# Find devices that have not been maintained in 30+ days
# AND have had >1 sensor anomaly in the last 7 days
# Join: Device → SensorReading → Alert + Device → MAINTAINED_BY

UNDERMAINTAINED_QUERY = """

"""

# Exercise B3: Create a Cypher query for shift handover report
# For outgoing shift (e.g., "day"), summarize:
# - Alerts created during shift
# - HVAC optimizations applied
# - Rooms that need follow-up
# Parameters: shift_start (datetime), shift_end (datetime)

SHIFT_REPORT_QUERY = """

"""

# Answers on next page (try first!)
```

Exercise Set B: Answer Key

Exercise B: Answer Key

```
-- B1: Alert Hotspots
MATCH (al:Alert)-[:AFFECTS]->(r:Room)
WHERE al.severity = 'HIGH'
AND al.ts > datetime() - duration({days: 7})
WITH r, count(al) AS alert_count,
max(al.ts) AS most_recent
WHERE alert_count > 2
RETURN r.id AS room_id, r.floor AS floor,
alert_count,
toString(most_recent) AS most_recent_alert
ORDER BY alert_count DESC;

-- B2: Under-maintained Devices
MATCH (d:Device)-[:RECORDED]->(sr:SensorReading)
-[:TRIGGERED]->(al:Alert)
WHERE al.ts > datetime() - duration({days: 7})
WITH d, count(al) AS anomaly_count
WHERE anomaly_count > 1
OPTIONAL MATCH (d)-[:MAINTAINED_BY]->(st:Staff)
<-[:PERFORMED]-(mj:MaintenanceJob)
WITH d, anomaly_count, max(mj.ts) AS last_maint
WHERE last_maint IS NULL
OR last_maint < datetime() - duration({days: 30})
RETURN d.id AS device_id, d.type AS device_type,
anomaly_count, toString(last_maint) AS last_maint
ORDER BY anomaly_count DESC;

-- B3: Shift Handover Report
MATCH (al:Alert)
WHERE al.ts >= datetime($shift_start)
AND al.ts <= datetime($shift_end)
WITH count(al) AS total_alerts,
sum(CASE WHEN al.severity='HIGH' THEN 1 ELSE 0 END)
AS high_alerts
MATCH (e:Event {type: 'hvac_optimization'})
WHERE e.ts >= datetime($shift_start)
AND e.ts <= datetime($shift_end)
WITH total_alerts, high_alerts, count(e) AS hvac_actions
```

```
RETURN total_alerts, high_alerts, hvac_actions,
total_alerts - high_alerts AS low_alerts;
```

Exercise Set C: ML Models

Exercise C: ML Model Challenges

```
# Exercise C1: Tune Isolation Forest contamination
# The default contamination=0.05 flags 5% of readings.
# Task: Test contamination values [0.01, 0.05, 0.10, 0.20]
# and plot precision vs recall using synthetic labeled data.

import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics import precision_score, recall_score

# Generate synthetic data: 90% normal, 10% anomaly
np.random.seed(42)
normal = np.random.normal([22, 55, 2, 0.6], [2,10,0.5,0.3],
size=(900, 4))
anomaly = np.random.uniform([30, 80, 4, 0], [40, 100, 6, 1],
size=(100, 4))
X = np.vstack([normal, anomaly])
y_true = np.array([0]*900 + [1]*100)

# TODO: Loop over contamination values and compute P/R
for contamination in [0.01, 0.05, 0.10, 0.20]:
model = IsolationForest(contamination=contamination,
random_state=42)
model.fit(X)
y_pred = (model.predict(X) == -1).astype(int)
# TODO: Compute and print precision and recall
pass


# Exercise C2: Add HVAC ramping constraint to LP
# Currently the LP optimizes each room independently.
# Add: |setpoint_r_t - setpoint_r_{t-1}| <= 2°C
# Hint: add two constraints per room using last_setpoint dict

def optimize_with_ramp(rooms, forecaster,
last_setpoints: dict) -> dict:
# TODO: Extend HVACOptimizer.optimize() with:
# prob += setpoints[r] <= last_setpoints.get(r, 22) + 2
# prob += setpoints[r] >= last_setpoints.get(r, 22) - 2
pass
```

Monitoring and Observability

Track agent performance, memory health, and system KPIs

Key Performance Indicators

KPI	Definition	Target	Alert Threshold	Cypher Query
Energy per Room (kWh/h)	Average hourly energy per occupied room	1.5-2.5 kWh	>4.0 kWh	MATCH ... RETURN avg(sr.kwh) WHERE sr.occupancy
Alert Resolution Time	Time from Alert created to MaintenanceJob closed	<2 hours	>4 hours	MATCH (al:Alert)-[:RESOLVED_BY]->(mj) RETURN avg(duration...)
Comfort Score	Fraction of occupied rooms within 20-26°C	>95%	<85%	MATCH ... WHERE occupancy AND 20<=temp<=26 RETURN count/total
Memory Write Latency	Time to write one event to all 4 layers	<200ms total	>500ms	Application-level timing in Python
KG Query P95 Latency	95th percentile Cypher query time	<200ms	>500ms	Application-level timing via time.perf_counter()
Anomaly Detection Rate	Fraction of readings flagged as anomalous	3-7%	>15% (over-detection)	MATCH (al:Alert) ... / MATCH (sr:SensorReading) ... RETURN count

Agent Health Monitor

monitor.py

```
# monitor.py — Agent health + KPI dashboard (CLI)

import time
import json
from datetime import datetime
from neo4j import GraphDatabase
import sqlite3

driver = GraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "hotel_mas_2024")
)

def measure_kg_latency() -> float:
"""Measure Cypher query round-trip time (ms)."""
start = time.perf_counter()
with driver.session() as s:
s.run("MATCH (n) RETURN count(n)").single()
return (time.perf_counter() - start) * 1000


def get_kpis() -> dict:
"""Compute all hotel KPIs from Neo4j."""
with driver.session() as s:
# Active alerts
alerts = s.run("""
MATCH (a:Alert)
WHERE a.ts > datetime() - duration({hours:1})
RETURN count(a) AS total,
sum(CASE WHEN a.severity='HIGH' THEN 1
```

```
ELSE 0 END) AS high_count
""").single()

# Energy KPI
energy = s.run("""
MATCH (sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours:1})
RETURN avg(sr.kwh) AS avg_kwh,
max(sr.kwh) AS max_kwh,
count(sr) AS readings
""").single()

# Comfort score
comfort = s.run("""
MATCH (sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours:1})
AND sr.occupancy = true
RETURN
sum(CASE WHEN 20 <= sr.temp_c <= 26
THEN 1 ELSE 0 END) AS in_range,
count(sr) AS total
""").single()

comfort_pct = (
(comfort['in_range'] / comfort['total'] * 100)
if comfort['total'] > 0 else 0
)
kg_ms = measure_kg_latency()

return {
"ts": datetime.now().isoformat(),
"alerts_1h": dict(alerts),
"energy_1h": {
"avg_kwh": round(energy['avg_kwh'] or 0, 2),
"max_kwh": round(energy['max_kwh'] or 0, 2),
"readings": energy['readings']
},
"comfort_score": round(comfort_pct, 1),
"kg_latency_ms": round(kg_ms, 1),
}


def print_dashboard():
kpis = get_kpis()
print(f"\n{'='*50}")
print(f"Smart Hotel MAS — {kpis['ts'][:19]}")
print(f"{'='*50}")
print(f" Alerts (1h): {kpis['alerts_1h']['total']} "
f"({kpis['alerts_1h']['high_count']} HIGH)")
print(f" Avg Energy: {kpis['energy_1h']['avg_kwh']} kWh")
print(f" Max Energy: {kpis['energy_1h']['max_kwh']} kWh")
print(f" Comfort Score: {kpis['comfort_score']}%")
print(f" KG Latency: {kpis['kg_latency_ms']} ms")

# Alerts
if kpis['alerts_1h']['high_count'] > 0:
print(f" ⚠ {kpis['alerts_1h']['high_count']} HIGH severity alerts!")
if kpis['comfort_score'] < 85:
print(f" ⚠ Comfort score below threshold: "
f"{kpis['comfort_score']}%")
if kpis['kg_latency_ms'] > 500:
print(f" ⚠ KG query latency high: "
f"{kpis['kg_latency_ms']}ms")
```

```
if __name__ == "__main__":
while True:
print_dashboard()
time.sleep(60) # refresh every minute
```

Key Insight: Monitor at least 3 dimensions: energy efficiency (are we saving kWh?), comfort quality (are guests within 20-26°C?), and system performance (is the KG responding in <200ms?). Set up alerts in PagerDuty or Slack when any KPI breaches its threshold.

Integration Testing

End-to-end test suite for the 4-memory architecture

Test Strategy

A complete MAS integration test validates that all 4 memory layers work together correctly, agent interactions produce expected outcomes, and edge cases (anomalies, Neo4j downtime, empty results) are handled gracefully.

Test Level	Scope	Tools	Run Frequency
Unit tests	Individual memory layer classes (WorkingMemory, EpisodicMemory, etc.)	pytest, pytest-mock	On every commit (CI)
Integration tests	Memory layers + Neo4j/ChromaDB (real containers)	pytest, docker-compose, testcontainers	On every PR merge
End-to-end tests	Full 5-agent CrewAI system with all memory layers	pytest, factory_boy, faker	Nightly
Load tests	API throughput, KG query latency under 200-room load	locust, k6	Weekly
Chaos tests	Neo4j failover, ChromaDB timeout, network partition	chaos-toolkit, pumba	Monthly

tests/test_memory_integration.py

```
# tests/test_memory_integration.py
# End-to-end tests for 4-memory architecture

import pytest
import json
from datetime import datetime

# ── Fixtures ──────────────────────────────────────────
@pytest.fixture(scope="session")
def working_mem():
"""Fresh WorkingMemory for test session."""
from memory.working import WorkingMemory
return WorkingMemory()


@pytest.fixture(scope="session")
def episodic_mem(tmp_path_factory):
"""EpisodicMemory backed by temp SQLite."""
from memory.episodic import EpisodicMemory
db = tmp_path_factory.mktemp("data") / "test_episodes.db"
mem = EpisodicMemory(str(db))
yield mem
mem.close()


@pytest.fixture(scope="session")
def semantic_mem():
"""SemanticMemory using ChromaDB (must be running)."""
from memory.semantic import SemanticMemory
return SemanticMemory(collection="test_hotel_memory")
```

```
@pytest.fixture(scope="session")
def kg_mem():
"""HotelKGMemory using Neo4j (must be running)."""
from memory.knowledge_graph import HotelKGMemory
return HotelKGMemory("TestAgent")


# ── L1 Tests ──────────────────────────────────────────
class TestWorkingMemory:
def test_set_get(self, working_mem):
working_mem.set("test_key", {"value": 42})
result = working_mem.get("test_key")
assert result == {"value": 42}

def test_default_on_missing(self, working_mem):
result = working_mem.get("nonexistent", default="fallback")
assert result == "fallback"

def test_summary_truncates_at_10(self, working_mem):
for i in range(15):
working_mem.set(f"key_{i}", i)
summary = working_mem.summary()
# Should not exceed 10 items in summary
item_count = summary.count("=")
assert item_count <= 10

def test_clear(self, working_mem):
working_mem.set("temp", "value")
working_mem.clear()
assert working_mem.get("temp") is None


# ── L2 Tests ──────────────────────────────────────────
class TestEpisodicMemory:
def test_store_and_recall(self, episodic_mem):
episodic_mem.store(
"TestAgent", "test_event",
{"room": "R101", "value": 24.5}
)
recent = episodic_mem.recall_recent(hours=1, limit=5)
assert len(recent) >= 1
found = any(
ep["type"] == "test_event"
for ep in recent
)
assert found, "Stored event not found in recall"

def test_filter_by_event_type(self, episodic_mem):
episodic_mem.store("TestAgent", "unique_type_xyz",
{"data": "test"})
results = episodic_mem.recall_recent(
hours=1, event_type="unique_type_xyz"
)
assert all(r["type"] == "unique_type_xyz"
for r in results)

def test_recall_by_room(self, episodic_mem):
episodic_mem.store("TestAgent", "room_event",
{"room": "R999", "temp": 22.0})
results = episodic_mem.recall_by_room("R999")
assert len(results) >= 1
```

```
# ── L4 KG Tests ──────────────────────────────────────────
class TestKGMemory:
def test_store_and_query_event(self, kg_mem):
kg_mem.store_event(
"test_sensor_read",
{"room": "R101", "temp_c": 23.5}
)
recent = kg_mem.recall_recent(hours=1, limit=5)
# Should have at least one event
assert isinstance(recent, list)

def test_cypher_executes(self, kg_mem):
result = kg_mem.cypher("RETURN 1 AS n")
assert result[0]["n"] == 1

def test_active_alerts_returns_list(self, kg_mem):
alerts = kg_mem.query_active_alerts()
assert isinstance(alerts, list)

def test_store_sensor_anomaly_creates_alert(self, kg_mem):
# First ensure room exists
kg_mem.cypher(
"MERGE (r:Room {id: 'R_TEST_999'}) "
"SET r.floor=9"
)
kg_mem.store_sensor_batch(
"R_TEST_999",
{"temp_c": 38.5, "humidity": 90, "kwh": 5.8},
anomaly=True
)
# Check alert was created
alerts = kg_mem.cypher("""
MATCH (al:Alert)-[:AFFECTS]->(r:Room {id:'R_TEST_999'})
RETURN count(al) AS cnt
""")
assert alerts[0]["cnt"] >= 1


# ── Cross-layer Integration ──────────────────────────────
class TestMemoryLayerIntegration:
def test_full_sensor_pipeline(self, working_mem,
episodic_mem, kg_mem):
"""Simulate a complete sensor processing cycle."""
room_id = "R_INTEGRATION_TEST"
reading = {
"temp_c": 25.0, "humidity": 60,
"kwh": 2.1, "occupancy": True
}

# L1: working memory
working_mem.set(f"latest_{room_id}", reading)
assert working_mem.get(f"latest_{room_id}") == reading

# L2: episodic store
episodic_mem.store("IntegrationTest",
"sensor_read", reading)
recent = episodic_mem.recall_recent(hours=1, limit=3)
assert len(recent) >= 1

# L4: KG store
kg_mem.store_sensor_batch(room_id, reading,
anomaly=False)
# If no exception, test passes
```

```
assert True
```

Running the Test Suite

docker-compose up -d neo4j chromadb redis # Start services pytest tests/ -v --tb=short # Run all tests pytest tests/test_memory_integration.py::TestKGMemory -v # KG tests only pytest tests/ --cov=memory --cov-report=html # Coverage report

Energy Analysis Deep Dive

Hotel energy patterns, tariff structures, and optimization strategies

Thai Hotel Energy Tariff Structure

Thai hotels operate under MEA (Metropolitan Electricity Authority) or PEA (Provincial Electricity Authority) Time-of-Use (TOU) tariffs. Understanding the tariff structure is critical for LP optimization.

Period	Hours	Rate (THB/kWh)	Strategy	LP Constraint
On-peak	09:00-21:00 weekdays	5.16	Minimize HVAC load, pre-cool	Objective weight: 5.16 × kWh
Off-peak	21:00-09:00 + weekends	2.64	Run HVAC freely, charge storage	Objective weight: 2.64 × kWh
Peak demand charge	Max 15-min demand (kW)	224.31/kW	Cap simultaneous HVAC starts	Σ_rooms load_r ≤ peak_capacity
Service charge	Fixed monthly	312.78	Fixed cost, not optimizable	Excluded from LP objective

energy_tariff.py

```
# energy_tariff.py — Thai MEA TOU tariff model

from datetime import datetime, time

class ThaiMEATariff:
"""
Thai Metropolitan Electricity Authority TOU tariff.
As of 2024 (rates in THB/kWh).
"""
ON_PEAK_RATE = 5.1583 # 09:00-21:00 weekdays
OFF_PEAK_RATE = 2.6369 # all other times
DEMAND_CHARGE = 224.31 # THB/kW (monthly peak)
SERVICE_CHARGE = 312.78 # THB/month (fixed)

ON_PEAK_START = time(9, 0)
ON_PEAK_END = time(21, 0)

@classmethod
def get_rate(cls, dt: datetime = None) -> float:
"""Return applicable rate for a given datetime."""
if dt is None:
dt = datetime.now()
# Weekends: always off-peak
if dt.weekday() >= 5:
return cls.OFF_PEAK_RATE
# Weekday: check hour
t = dt.time()
if cls.ON_PEAK_START <= t < cls.ON_PEAK_END:
return cls.ON_PEAK_RATE
return cls.OFF_PEAK_RATE

@classmethod
def daily_cost(cls, hourly_kwh: list) -> float:
"""
Compute daily electricity cost.
hourly_kwh: list of 24 floats (one per hour)
"""
```

```
total = 0.0
now = datetime.now().replace(hour=0, minute=0)
for h, kwh in enumerate(hourly_kwh):
dt = now.replace(hour=h)
total += kwh * cls.get_rate(dt)
return round(total, 2)

@classmethod
def monthly_estimate(cls,
avg_daily_kwh: float,
peak_demand_kw: float) -> dict:
"""
Estimate monthly bill for a hotel floor.
"""
# Assume 22 weekdays, 8 weekend days
daily = avg_daily_kwh * cls.get_rate()
monthly_energy = daily * 30
monthly_demand = peak_demand_kw * cls.DEMAND_CHARGE
return {
"energy_cost_thb": round(monthly_energy, 2),
"demand_cost_thb": round(monthly_demand, 2),
"service_charge_thb": cls.SERVICE_CHARGE,
"total_thb": round(
monthly_energy + monthly_demand
+ cls.SERVICE_CHARGE, 2
)
}

@classmethod
def savings_from_setback(cls,
n_rooms: int,
setback_temp: float = 28.0,
normal_temp: float = 24.0,
setback_hours: int = 8) -> dict:
"""
Estimate energy/cost savings from temperature setback
in unoccupied rooms during night hours.
"""
# Physics: COP ~3.0 for modern VRF HVAC
COP = 3.0
# kWh saved per degree-hour per room
# Approx: 0.1 kWh per °C per hour per room
KWHR_PER_DEGREE = 0.1
delta_t = setback_temp - normal_temp
kwh_saved_total = (delta_t * KWHR_PER_DEGREE
* setback_hours * n_rooms / COP)
cost_saved = kwh_saved_total * cls.OFF_PEAK_RATE
return {
"delta_t_c": delta_t,
"kwh_saved": round(kwh_saved_total, 1),
"thb_saved_nightly": round(cost_saved, 2),
"thb_saved_monthly": round(cost_saved * 30, 2),
"rooms": n_rooms
}

# ── Example ──────────────────────────────────────────
if __name__ == "__main__":
tariff = ThaiMEATariff()
current_rate = tariff.get_rate()
print(f"Current rate: {current_rate} THB/kWh")

# Estimate savings from nightly setback (200 rooms)
savings = tariff.savings_from_setback(
```

```
n_rooms=200, setback_temp=28.0,
normal_temp=22.0, setback_hours=8
)
print(f"Nightly setback savings: "
f"{savings['kwh_saved']} kWh = "
f"{savings['thb_saved_nightly']} THB/night")
print(f"Monthly savings: {savings['thb_saved_monthly']} THB")

# Bill estimate for one floor (40 rooms)
bill = tariff.monthly_estimate(
avg_daily_kwh=40 * 2.0, # 2 kWh/room/day
peak_demand_kw=40 * 3.5 # 3.5 kW per room peak
)
print(f"Monthly bill (1 floor): {bill['total_thb']} THB")
```

Energy Savings Calculation

With 200 rooms and 8-hour nightly setback (22°C → 28°C): kWh saved = ΔT × 0.1 kWh/°C/h/room × 8h × 200 rooms / COP(3.0) = 6°C × 0.1 × 8 × 200 / 3.0 = 320 kWh/night Cost saved = 320 kWh × 2.64 THB/kWh = 845 THB/night Monthly: = 845 × 30 = 25,350 THB/month (~$700 USD) With LP optimization (occupancy-aware setpoints) vs fixed setpoint: Typical additional saving: 15-25% on top of manual scheduling = 4,000-6,000 THB/month for a 200-room hotel

Energy Analysis Cypher Queries

Energy Analysis Queries

```
// Energy analysis queries for hotel operations team

// 1. Hourly energy profile (today)
MATCH (sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours: 24})
WITH sr.ts.hour AS hour, avg(sr.kwh) AS avg_kwh,
sum(sr.kwh) AS total_kwh,
count(sr) AS readings
RETURN hour, round(avg_kwh * 100)/100 AS avg_kWh_per_room,
round(total_kwh * 100)/100 AS total_kWh,
readings
ORDER BY hour;

// 2. Peak demand windows (15-minute intervals)
MATCH (sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours: 24})
WITH sr.ts.hour AS h,
(toInteger(sr.ts.minute) / 15) AS interval_15min,
sum(sr.kwh) AS interval_kwh
RETURN h * 4 + interval_15min AS interval_id,
toString(h) + ':' +
toString(interval_15min * 15) AS time_label,
round(interval_kwh * 100)/100 AS total_kwh_15min
ORDER BY interval_kwh DESC
LIMIT 10;

// 3. Energy efficiency by room type
MATCH (r:Room)-[:HAS_DEVICE]->(d:Device {type:'SENSOR'})
-[:RECORDED]->(sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours: 24})
AND sr.occupancy = true
WITH r.type AS room_type,
avg(sr.kwh) AS avg_kwh_occupied,
count(sr) AS readings
RETURN room_type,
round(avg_kwh_occupied * 100)/100 AS avg_kWh,
readings
```

```
ORDER BY avg_kwh_occupied;

// 4. Wasted energy: empty rooms consuming too much
MATCH (r:Room)-[:HAS_DEVICE]->(d:Device {type:'SENSOR'})
-[:RECORDED]->(sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours: 4})
AND sr.occupancy = false
AND sr.kwh > 2.5 -- threshold for empty room
WITH r.id AS room, r.floor AS floor,
avg(sr.kwh) AS avg_kwh, count(sr) AS readings
WHERE avg_kwh > 2.5
RETURN room, floor, round(avg_kwh * 100)/100 AS avg_kWh,
readings AS samples
ORDER BY avg_kwh DESC;

// 5. Energy savings from HVAC optimizations
MATCH (e:Event {type: 'hvac_optimization'})
WHERE e.ts > datetime() - duration({days: 7})
WITH e, apoc.convert.fromJsonMap(e.data) AS d
RETURN sum(toFloat(d.energy_saved)) AS total_kwh_saved,
count(e) AS optimization_actions,
avg(toFloat(d.energy_saved)) AS avg_saving_per_action
;
```

Week 10 Preview: Advanced Extensions

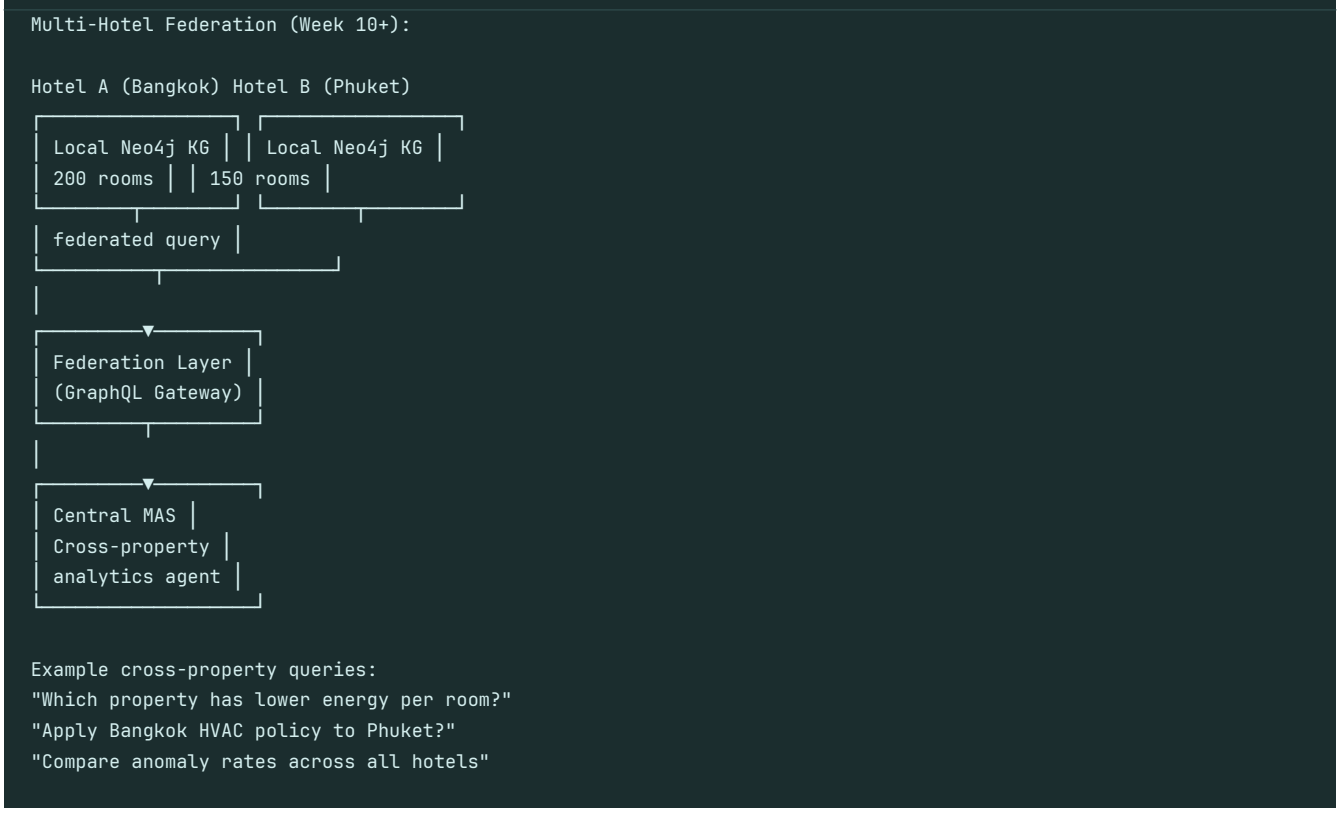
What comes next — MQTT, real-time dashboard, multi-hotel federation

Advanced Topics Roadmap

Topic	Description	Technology	Estimated Effort
Real-time MQTT	Replace simulated reads with live IoT sensors via MQTT	Eclipse Mosquitto, paho-mqtt, Raspberry Pi	4 hours
Cloud Deployment	Deploy to Neo4j Aura + Render.com, environment hardening	Neo4j Aura, Render.com, GitHub Actions CI/CD	8 hours
Streamlit Dashboard	Live hotel manager chat interface with GraphRAG	Streamlit, Plotly, websockets	6 hours
RL Production Mode	Shadow mode → A/B test → full RL policy deployment	Stable-Baselines3, MLflow, Evidently AI	16 hours
Multi-hotel Federation	Connect multiple hotel KGs, cross-property analytics	Neo4j Federation, GraphQL, data contracts	24 hours
Predictive Maintenance	Predict device failures before they happen	PyMC3, survival analysis, ONNX runtime	12 hours
Dynamic Pricing Agent	Adjust room rates based on demand + energy costs	CrewAI, Anthropic, PuLP, Booking.com API	16 hours
Guest Preference KG	Learn and apply individual guest preferences	Neo4j, ChromaDB, LangChain memory	8 hours

Multi-Hotel Federation Architecture

Multi-Hotel Federation



```
Federation query (Cypher Federation):
CALL {
USE neo4j-bangkok
MATCH (r:Room) RETURN r, 'BKK' AS hotel
UNION ALL
USE neo4j-phuket
MATCH (r:Room) RETURN r, 'PHU' AS hotel
}
WITH hotel, avg(r.kwh) AS avg_energy
RETURN hotel, avg_energy ORDER BY avg_energy
```

Week 10 Starter: MQTT Integration

This is the highest-impact extension: replacing simulated sensor data with real IoT readings makes the system production-ready. A Raspberry Pi 4 (~$50) can bridge 200 ESP32 sensors via Zigbee/WiFi.

Week 10: MQTT Real-time Integration

```
# Week 10 starter: Real ESP32 sensor data format
# ESP32 publishes to: hotel/floor/{n}/room/{id}/sensor

# Payload format (JSON, sent every 30 seconds):
{
"ts": "2024-11-15T14:32:05.123Z", # UTC ISO 8601
"device_id": "SENSOR_R301",
"room_id": "R301",
"floor": 3,
"temp_c": 23.4, # DS18B20 temperature sensor
"humidity": 58.2, # DHT22 humidity sensor
"occupancy": true, # PIR motion sensor (30s timeout)
"co2_ppm": 650, # MH-Z19B CO2 sensor (optional)
"lux": 320, # BH1750 light sensor (optional)
"kwh": 2.15, # Pzem-004T energy meter
"firmware": "1.4.2" # For OTA update tracking
}

# MQTT Subscribe (Python):
import paho.mqtt.client as mqtt
import json

client = mqtt.Client()
client.connect("localhost", 1883)

def on_message(client, userdata, msg):
data = json.loads(msg.payload)
room_id = data['room_id']
# → process through SensorAgent pipeline
process_reading(room_id, data)

client.subscribe("hotel/floor/+/room/+/sensor", qos=1)
client.on_message = on_message
client.loop_forever()

# Hardware BOM per room (approximate costs in USD):
# ESP32-WROOM-32D: $3.50 (main MCU)
# DS18B20 temp: $1.20 (waterproof, ±0.5°C)
# DHT22 humidity: $2.80 (±2% RH, ±0.5°C)
# PIR HC-SR501: $0.80 (occupancy)
# PZEM-004T energy: $8.00 (kWh meter, CT clamp)
# Total per room: ~$16.30 (qty 200: ~$3,260)
# Raspberry Pi 4 hub: $55.00 (central MQTT broker)
```

Key Insight: The total hardware cost for a 200-room hotel IoT deployment is approximately $3,315 USD. With energy savings of $700/month from the MAS optimization, the hardware pays for itself in under 5 months. Ongoing cloud costs add $250/month — still net positive from month 1.

Your MAS Skill Progression

Week	Topic	Key Skill	Capstone Project
7	MAS Design Thinking	Agent role decomposition, message flows	Design 5-agent system for chosen domain
8	MAS Environment Setup	Docker, Neo4j, ChromaDB, Redis	Running services + connectivity tests
9	MAS KG Coding (this workshop)	4-memory architecture, LP, RL, Isolation Forest	Smart Hotel MAS end-to-end system
10	MAS Production	MQTT, FastAPI, Streamlit, CI/CD	Deploy hotel MAS to cloud with dashboard
11	MAS Fine-tuning	RL policy convergence, LP tuning, prompt engineering	Optimize MAS for >30% energy reduction
12	MAS Capstone	Full production system	Your domain: custom MAS from scratch

Reference Section

Complete Cypher schema, all GraphRAG queries, troubleshooting guide

Full Cypher Schema

Complete setup script including all constraints, indexes, and sample relationship creation for the hotel domain.

complete_schema.cypher

```
// ============================================
// Hotel MAS — Complete Cypher Schema Setup
// Run once after docker-compose up
// ============================================

// --- Node Constraints ---
CREATE CONSTRAINT IF NOT EXISTS FOR (r:Room)
REQUIRE r.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (d:Device)
REQUIRE d.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (s:Staff)
REQUIRE s.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (g:Guest)
REQUIRE g.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (a:Agent)
REQUIRE a.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (sk:Skill)
REQUIRE sk.name IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (fl:Floor)
REQUIRE fl.number IS UNIQUE;

// --- Performance Indexes ---
CREATE INDEX IF NOT EXISTS FOR (sr:SensorReading)
ON (sr.ts);
CREATE INDEX IF NOT EXISTS FOR (al:Alert)
ON (al.ts, al.severity);
CREATE INDEX IF NOT EXISTS FOR (e:Event)
ON (e.type, e.ts);
CREATE INDEX IF NOT EXISTS FOR (mj:MaintenanceJob)
ON (mj.status, mj.priority);
CREATE INDEX IF NOT EXISTS FOR (ee:EnergyEvent)
ON (ee.ts);

// --- Sample: Create Floors ---
UNWIND range(1, 5) AS n
MERGE (f:Floor {number: n, name: 'Floor ' + n});

// --- Link Rooms to Floors ---
MATCH (r:Room), (f:Floor)
WHERE r.floor = f.number
MERGE (r)-[:ON_FLOOR]->(f);

// --- Sample Staff ---
MERGE (s1:Staff {id: 'ENG001'})
SET s1.name = 'Somchai K.', s1.role = 'HVAC Engineer',
s1.shift = 'day', s1.available = true;
MERGE (s2:Staff {id: 'ENG002'})
SET s2.name = 'Nattawut P.', s2.role = 'Electrician',
s2.shift = 'night', s2.available = true;

// --- Skills ---
MERGE (sk1:Skill {name: 'HVAC_REPAIR'});
MERGE (sk2:Skill {name: 'ELECTRICAL'});
MERGE (sk3:Skill {name: 'PLUMBING'});
```

```
// --- Staff-Skill edges ---
MATCH (s:Staff {id: 'ENG001'}), (sk:Skill {name: 'HVAC_REPAIR'})
MERGE (s)-[:SKILLED_IN]->(sk);
MATCH (s:Staff {id: 'ENG002'}), (sk:Skill {name: 'ELECTRICAL'})
MERGE (s)-[:SKILLED_IN]->(sk);

// --- Register Agents ---
MERGE (ag1:Agent {id: 'SensorAgent'})
SET ag1.role = 'IoT Monitor', ag1.status = 'active';
MERGE (ag2:Agent {id: 'EnergyAgent'})
SET ag2.role = 'LP Optimizer', ag2.status = 'active';
MERGE (ag3:Agent {id: 'MemoryAgent'})
SET ag3.role = 'KG Steward', ag3.status = 'active';
MERGE (ag4:Agent {id: 'AlertAgent'})
SET ag4.role = 'Anomaly Detector', ag4.status = 'active';
MERGE (ag5:Agent {id: 'ReportAgent'})
SET ag5.role = 'GraphRAG Reporter', ag5.status = 'active';
```

All GraphRAG Queries

These production-ready Cypher queries cover the most common hotel intelligence use cases.

hotel_graphrag_queries.cypher

```
// 1. Floor energy summary (last 1 hour)
MATCH (r:Room)-[:HAS_DEVICE]->(d:Device {type:'SENSOR'})
-[:RECORDED]->(sr:SensorReading)
WHERE r.floor = 3
AND sr.ts > datetime() - duration({hours: 1})
RETURN r.id AS room,
avg(sr.temp_c) AS avg_temp_c,
avg(sr.kwh) AS avg_kwh,
count(sr) AS total_readings
ORDER BY avg_kwh DESC;

// 2. Active alerts last 4 hours
MATCH (a:Alert)-[:AFFECTS]->(r:Room)
WHERE a.ts > datetime() - duration({hours: 4})
RETURN r.id AS room, a.type AS alert,
a.severity AS severity,
toString(a.ts) AS triggered_at
ORDER BY a.ts DESC LIMIT 20;

// 3. Rooms with >3 HVAC actions today
MATCH (e:Event)-[:INVOLVES]->(d:Device {type:'HVAC'})
<-[:HAS_DEVICE]-(r:Room)
WHERE e.type = 'hvac_optimization'
AND e.ts > datetime() - duration({hours: 24})
WITH r, count(e) AS actions
WHERE actions > 3
RETURN r.id AS room, r.floor AS floor, actions
ORDER BY actions DESC;

// 4. Agent action history (last 10 per agent)
MATCH (a:Agent)-[:PERFORMED]->(e:Event)
WITH a, e ORDER BY e.ts DESC
WITH a, collect(e)[0..10] AS recent_events
UNWIND recent_events AS ev
RETURN a.id AS agent, ev.type AS action,
toString(ev.ts) AS when
ORDER BY ev.ts DESC;

// 5. Cascade: alert -> device -> room -> floor
MATCH (al:Alert)-[:CAUSED_BY]->(d:Device)
<-[:HAS_DEVICE]-(r:Room)
```

```
-[:ON_FLOOR]->(f:Floor)
WHERE al.severity = 'HIGH'
RETURN f.name AS floor, r.id AS room,
d.type AS device, al.type AS alert,
toString(al.ts) AS ts
ORDER BY al.ts DESC;

// 6. Best-performing rooms (low energy, high occupancy)
MATCH (r:Room)-[:HAS_DEVICE]->(d:Device {type:'SENSOR'})
-[:RECORDED]->(sr:SensorReading)
WHERE sr.ts > datetime() - duration({hours: 24})
WITH r, avg(sr.kwh) AS avg_energy,
sum(CASE WHEN sr.occupancy THEN 1 ELSE 0 END) AS occ_cnt,
count(sr) AS total
WITH r, avg_energy, occ_cnt,
toFloat(occ_cnt)/total AS occ_rate
WHERE occ_rate > 0.5 AND avg_energy < 2.0
RETURN r.id AS room, round(avg_energy*100)/100 AS kWh,
round(occ_rate*100)/100 AS occupancy_rate
ORDER BY avg_energy ASC LIMIT 10;

// 7. Maintenance staff availability
MATCH (st:Staff)-[:SKILLED_IN]->(sk:Skill)
WHERE st.available = true
AND sk.name IN ['HVAC_REPAIR','ELECTRICAL']
OPTIONAL MATCH (st)<-[:ASSIGNED_TO]-(mj:MaintenanceJob
{status: 'in_progress'})
RETURN st.id AS staff_id, st.name AS name,
st.shift AS shift,
collect(sk.name) AS skills,
count(mj) AS active_jobs
ORDER BY active_jobs ASC;
```

Troubleshooting Guide

Error	Cause	Fix
Connection refused port 7687	Neo4j not running	docker-compose up neo4j; wait 30s
ChromaDB connection refused	ChromaDB not running	docker-compose up chromadb
ConstraintValidationFailed	Duplicate node creation	Use MERGE instead of CREATE
No space left on device	Docker volume full	docker volume prune
PuLP solver not found	CBC not installed	pip install pulp; brew install glpk
prophet not installed	Missing dependency	pip install prophet (requires pystan)
ANTHROPIC_API_KEY not set	Missing env var	export ANTHROPIC_API_KEY=sk-ant-...
CrewAI LLM error	Model not found	Use llm='claude-3-5-sonnet-20241022'
GDS procedure not found	GDS plugin missing	Set NEO4J_PLUGINS in docker-compose.yml
IsolationForest not fitted	No training data	Run seed_hotel.py first, then fit_from_neo4j()

What to Build Next

Week 10 Extensions

1. Add real-time MQTT subscription (Eclipse Mosquitto) — replace simulated sensor reads 2. Deploy to cloud: Neo4j Aura + Render for agent backends 3. Add a Streamlit dashboard: live GraphRAG chat interface for hotel managers 4. Implement the RL agent in production: shadow mode → A/B test → full deployment 5. Connect to room booking system: add :Booking nodes → dynamic pricing agents

Quick Reference Card

Key Environment Variables

Environment Setup Reference

```
# .env — Copy to project root
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=hotel_mas_2024
CHROMA_HOST=localhost
CHROMA_PORT=8001
ANTHROPIC_API_KEY=sk-ant-YOUR_KEY_HERE
REDIS_URL=redis://localhost:6379/0

# Verify connections
python -c "from neo4j import GraphDatabase; d=GraphDatabase.driver('bolt://localhost:7687',
auth=('neo4j','hotel_mas_2024')); print('Neo4j OK'); d.close()"

python -c "import chromadb; c=chromadb.HttpClient('localhost',8001); print('ChromaDB OK:', c.heartbeat())"

python -c "import redis; r=redis.Redis(); print('Redis OK:', r.ping())"

# Useful docker commands
docker-compose ps # check service status
docker-compose logs neo4j --tail=50 # view Neo4j logs
docker-compose restart neo4j # restart a service
docker exec -it smart-hotel-neo4j-1 cypher-shell -u neo4j -p hotel_mas_2024 "MATCH (n) RETURN count(n);" # quick
count
```

Project File Structure

Project Structure

```
smart_hotel_mas/
├── docker-compose.yml # Neo4j + ChromaDB + Redis
├── .env # Environment variables
├── requirements.txt # Python dependencies
│
├── seed_hotel.py # One-time database seeder
│
├── memory/
│ ├── __init__.py
│ ├── working.py # L1: WorkingMemory class
│ ├── episodic.py # L2: EpisodicMemory (SQLite)
│ ├── semantic.py # L3: SemanticMemory (ChromaDB)
│ └── knowledge_graph.py # L4: HotelKGMemory (Neo4j)
│
├── agents/
│ ├── sensor_agent.py # IoT reading + anomaly check
│ ├── energy_agent.py # LP + ARIMA optimization
│ ├── memory_agent.py # Memory steward
│ ├── alert_agent.py # Alert creation + dispatch
│ └── report_agent.py # GraphRAG + NL reporting
│
├── models/
│ ├── hvac_optimizer.py # PuLP LP formulation
│ ├── occupancy_forecaster.py # Prophet time series
│ ├── rl_hvac_env.py # Gymnasium environment
│ └── anomaly_detector.py # Isolation Forest
│
├── cypher/
│ ├── schema.cypher # Constraints + indexes
│ ├── graphrag_queries.cypher # Operational queries
```

```
│   └── seed_data.cypher # Sample data
│
└── main.py # Entry point: run full MAS
```

Learning Resources

Resource	Topic	URL
Neo4j Docs	Cypher query language reference	neo4j.com/docs/cypher-manual
ChromaDB Docs	Vector database + embeddings	docs.trychroma.com
CrewAI Docs	Multi-agent orchestration	docs.crewai.com
PuLP Docs	Linear programming in Python	coin-or.github.io/pulp
Prophet Docs	Time series forecasting	facebook.github.io/prophet
Stable-Baselines3	Deep RL algorithms	stable-baselines3.readthedocs.io
Isolation Forest	sklearn anomaly detection	scikit-learn.org/stable/modules/outlier_detection
GraphRAG Paper	LLM + Knowledge Graph retrieval	arxiv.org/abs/2404.16130